AWS 환경에서 Helm & Shell 스크립트를 활용한 Kubernetes 자동화 배포 시스템 구축

TEAM 에코 클라우드
김선호 외 3명



CONTENTS

1 팀 소개

2 프로젝트 소개

3 개발 환경

4 기술 소개

5 활용 방안 및 기대 효과

6 느낀점

01
팀 소개: Team Echo Cloud!!!



김선호 (팀장)

EKS
SHELL
HELM



팀원 A

MONITORING
SHELL
HELM



팀원 B

NODE.JS
SHELL
HELM



팀원 C

TERRAFORM
SHELL
HELM

02

프로젝트 소개: 선정 배경

01

반복적이고 비효율적인 Kubernetes 배포 작업을 자동화할 필요성이 증가

02

운영 비용 절감과 배포 시간 단축을 위한 체계적 시스템 필요

03

모니터링 및 오류 대응이 실시간으로 이루어져야 운영 안정성을 확보할 수 있음

02

프로젝트 소개: 프로젝트 목적



운영 비용 절감 &
배포 시간 단축



사용자별 배포
자동화

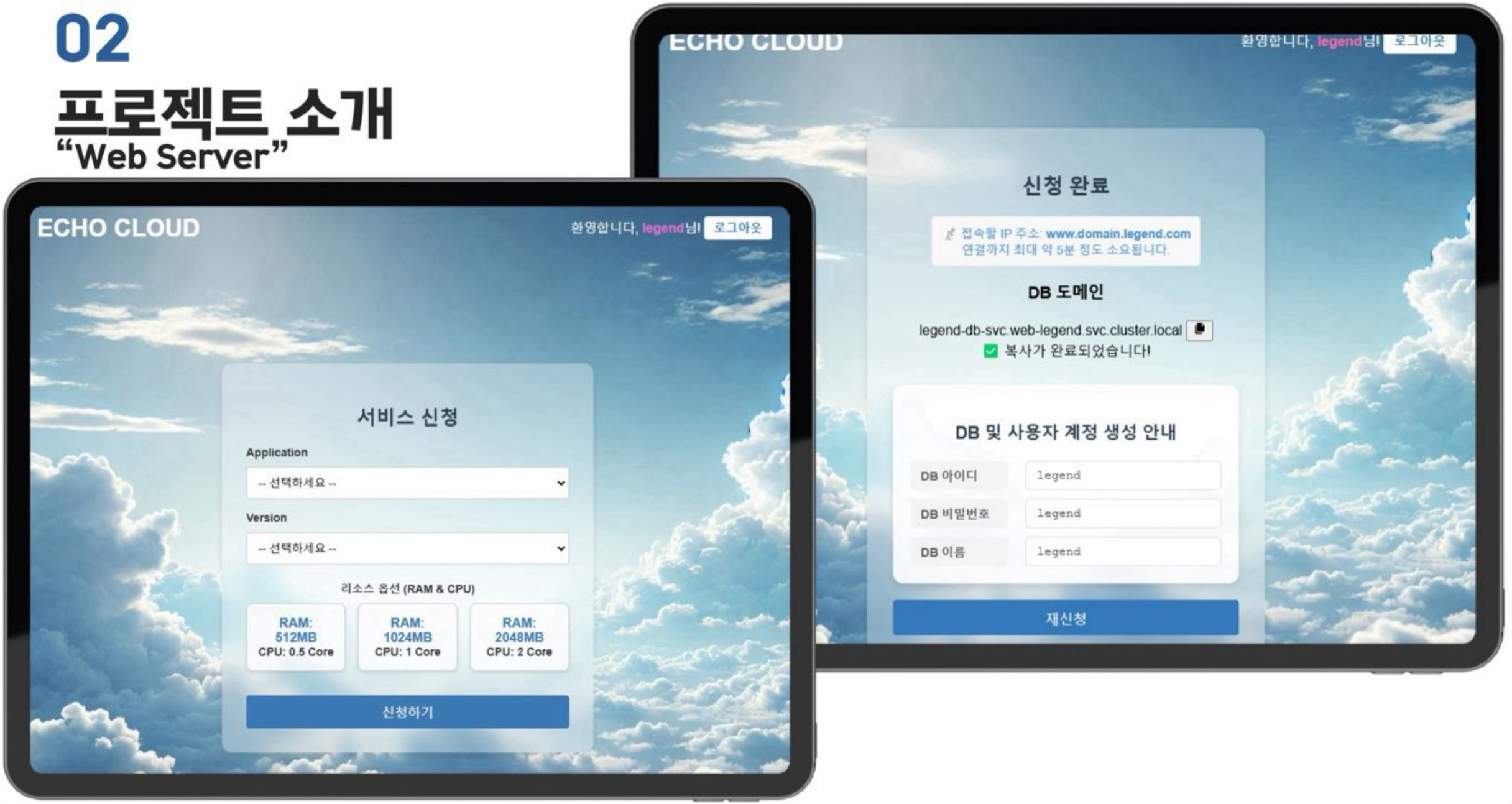


모니터링 및 문제
대응 체계 강화

02

프로젝트 소개

“Web Server”

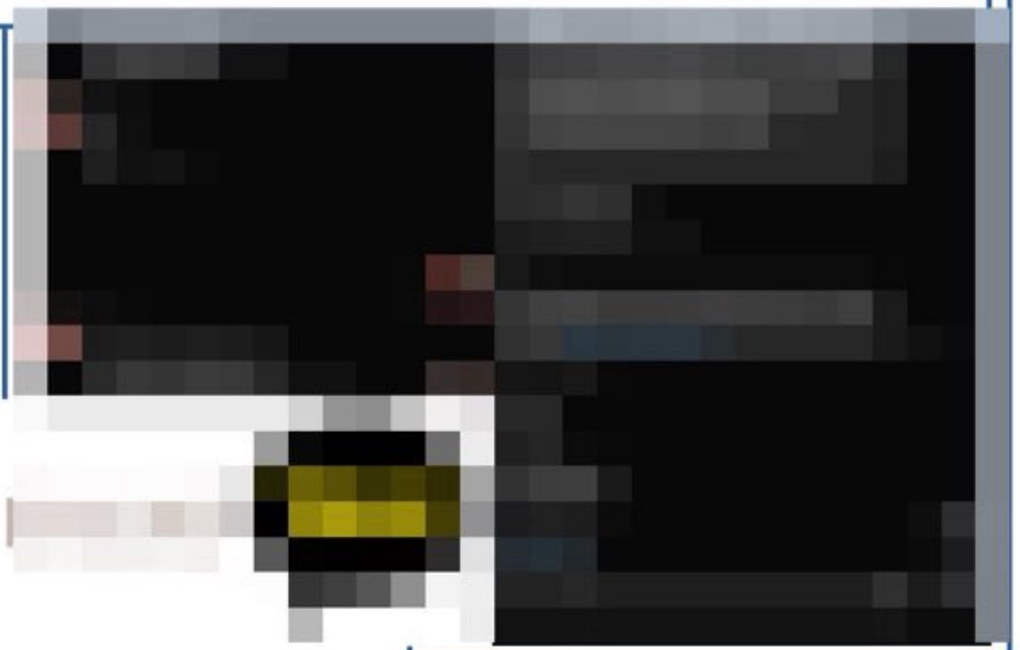


03
개발 환경: 협업 도구

Jira

Confluence





03
개발 환경
“Architecture”

amazon web services

GitHub

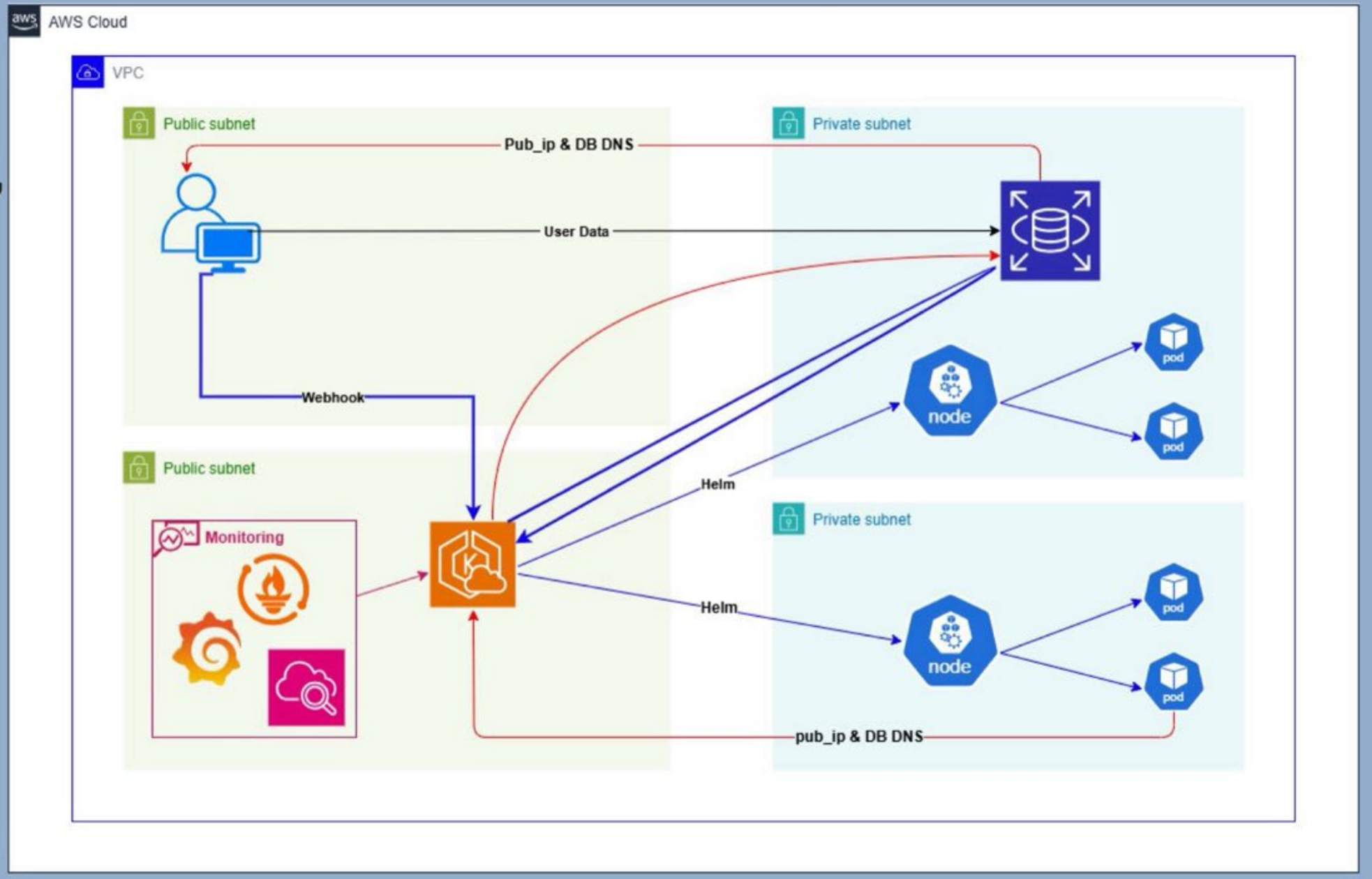
node.js

vmware

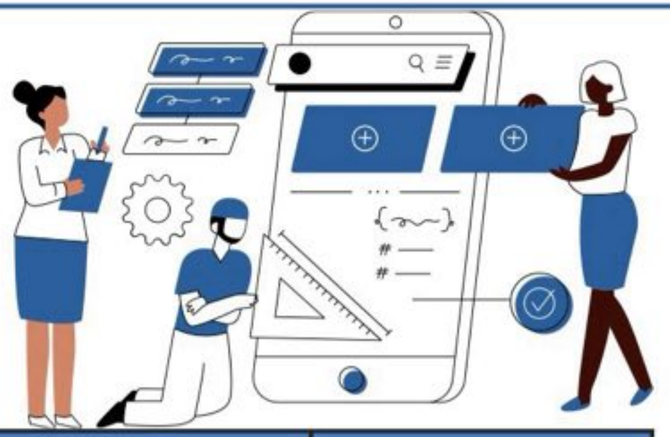
HELM

Terraform

MySQL



03
개발 환경



	Public		Private		VMware
Instance	Bastion	WebSrv	RDS	Worker	Controller
CPU	1vCPU	1vCPU	2vCPU	4vCPU	4 Core
RAM	1GB	1GB	1GB	8GB	8GB

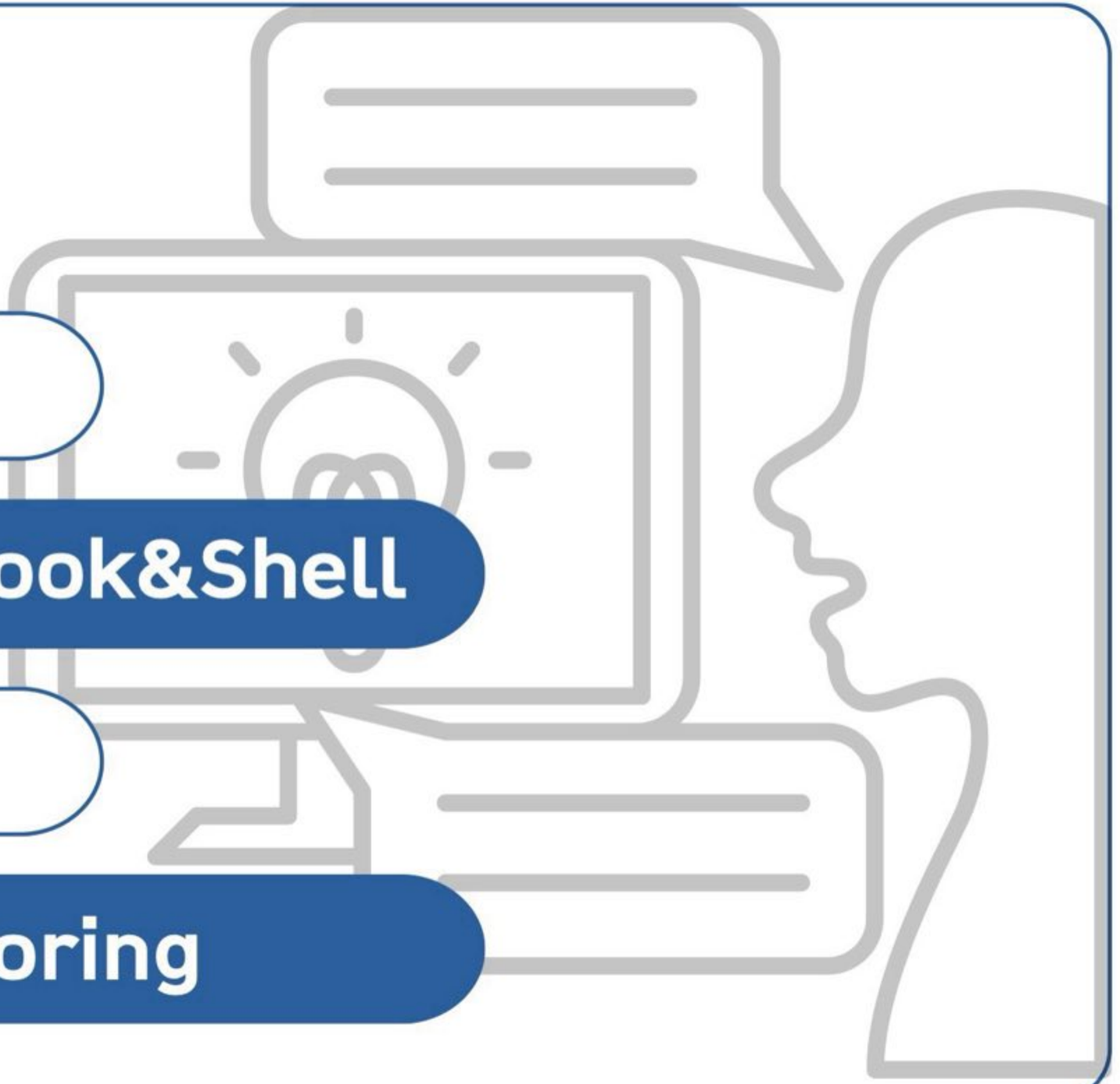
04
기술 소개

Terraform

Webhook&Shell

Helm

Monitoring



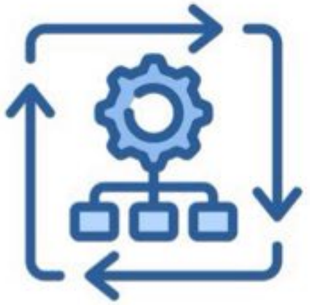
04
기술 소개: Terraform



Infrastructure as Code (IaC)



일관성 및 재사용성



초기 설정 자동화



비용 최적화

04
기술 소개: Webhook



이벤트 기반 자동화

"WebSrv .env"

```
WEBHOOK_URL=[redacted] Webhook을 받는 상대방 IP /webhook
WEBHOOK_SECRET= Secret_Key 입력
```

"Bastion .env"

```
WEBHOOK_SECRET= 입력했던 Secret_Key 입력
SCRIPT_PATH=/lab/main.sh
```

유연한 포트 설정

보안 강화 (Secret Key)

맞춤형 스크립트 실행

04

기술 소개: Shell

- fetch_users.sh
- process_user.sh
- helm_setup.sh
- deployment_check.sh
- external_ip.sh



일관된 환경과 운영 효율성을 제공하는 절차형 스크립트

세분화

자동화

절차형

04

기술 소개: helm_setup.sh

```
# deployment.yaml 및 service.yaml 복사
echo "[$user_id]의 템플릿 디렉토리에 deployment.yaml 및 service.yaml을 복사합니다."
cp deployment.yaml "$user_id/templates/deployment.yaml"
cp service.yaml "$user_id/templates/service.yaml"
echo "[$user_id]에 사용자 정의 템플릿 파일을 복사했습니다."

# values.yaml 템플릿 파일 설정
echo "[$user_id]의 values.yaml 파일을 설정합니다."
cp values.yaml "$user_id/values.yaml"
sed -i "s|APP|$app|g" "$user_id/values.yaml"
sed -i "s|VERSION|$ver|g" "$user_id/values.yaml"
sed -i "s|USER_ID|$user_id|g" "$user_id/values.yaml"
sed -i "s|CPU|$cpu|g" "$user_id/values.yaml"
sed -i "s|RAM|$ram|g" "$user_id/values.yaml"
echo "[$user_id]의 values.yaml 파일이 성공적으로 업데이트되었습니다."
```

복사

치환

04

기술 소개: deployment_check.sh

```bash
#!/bin/bash

current_hash=$(find "$user_id/templates" "$user_id/values.yaml" -type f -exec sha256sum {} \; | sort | sha256sum | awk '{print $1}')
hash_file="$user_id/.last_deploy_hash"

if [[ -f "$hash_file" ]]; then
  previous_hash=$(cat "$hash_file")
else
  previous_hash=""
fi

if [[ "$current_hash" == "$previous_hash" ]]; then
  echo "[$user_id]에는 변경된 내용이 없습니다. 배포를 건너뜁니다."
else
  echo "[$user_id]에 변경사항이 감지되었습니다. Helm 배포를 진행합니다."
  if helm upgrade --install "$user_id-helm" "./$user_id" --namespace "web-$user_id" --create-namespace; then
    echo "$(date): [$user_id] | 애플리케이션: [$app] | 버전: [$ver] 배포가 성공적으로 완료되었습니다." >> deployment.log
    echo "[$user_id]의 Helm 배포가 완료되었습니다."
    echo "$current_hash" > "$hash_file"
  else
    echo "$(date): [$user_id] | 애플리케이션: [$app] | 버전: [$ver] 배포에 실패했습니다." >> deployment.log
    echo "[$user_id]의 Helm 배포 중 오류가 발생했습니다."
  fi
fi
```

Hash값 비교 & 배포

04

기술 소개: HELM

롤백 및 업그레이드

Kubernetes 클러스터에서 패키지 관리를 제공하는 오픈소스 도구

애플리케이션의 배포, 업그레이드, 롤백을 간소화하고 일관된 환경을 보장

패키지 관리

간편한 배포

버전 관리

04
기술 소개: HELM

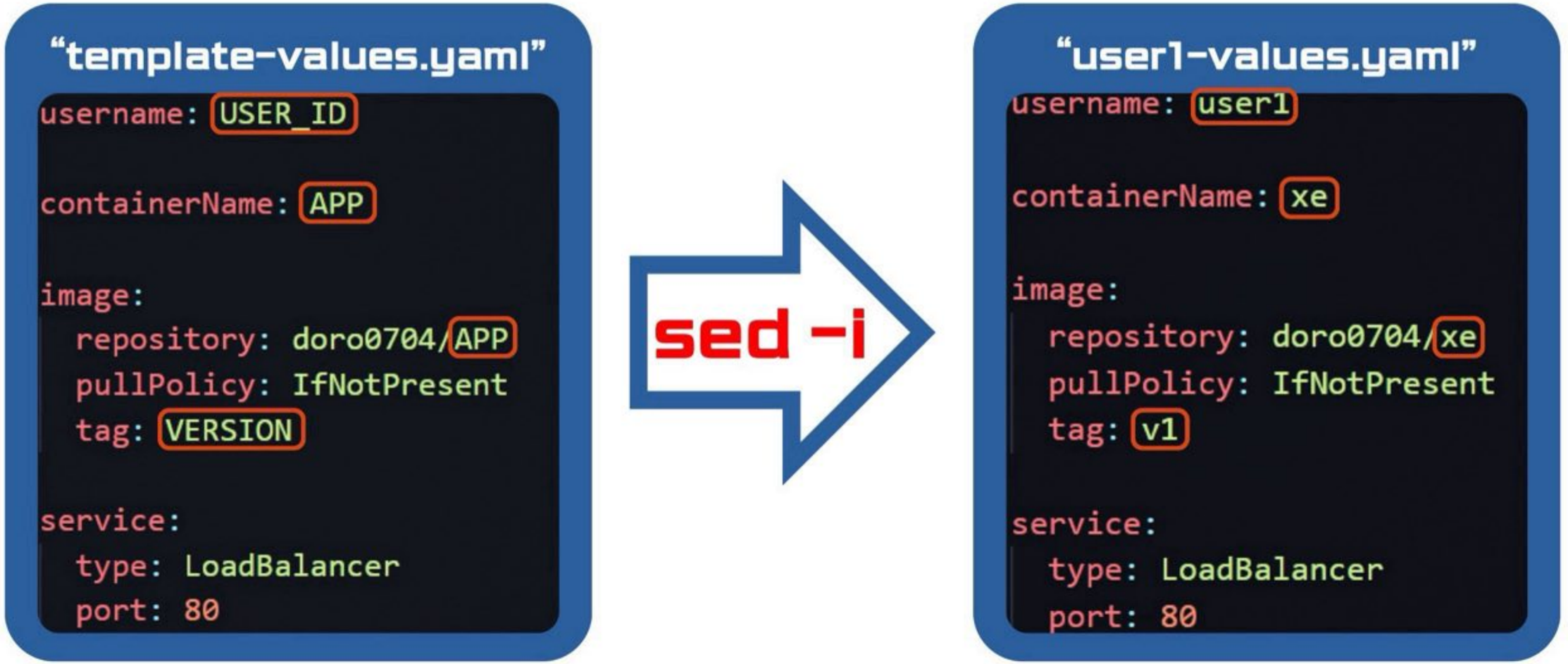
생성

```
.
├── charts
├── Chart.yaml
├── templates
│   ├── deployment.yaml
│   ├── _helpers.tpl
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── NOTES.txt
│   ├── serviceaccount.yaml
│   ├── service.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: "{{ .Values.username }}-{{ .Values.db.containerName }}"
  labels:
    app: "{{ .Chart.Name }}-db"
spec:
  replicas: 1
  selector:
    matchLabels:
      app: "{{ .Chart.Name }}-db"
```

변수 사용

04
기술 소개: helm values.yaml



04

기술 소개: helm 추가적으로 알게된 점

작업	HELM	Kubectl
배포	✅ helm install	✅ kubectl apply -f
삭제	✅ helm uninstall	✅ kubectl delete -f
업데이트	✅ helm upgrade	✅ kubectl apply -f
롤백	✅ helm rollback	❌ 지원하지 않음
로그 확인	❌ 지원하지 않음	✅ kubectl logs
상태 확인	❌ 지원하지 않음	✅ kubectl get pods
리소스 상세 확인	❌ 지원하지 않음	✅ kubectl describe
클러스터 관리	❌ 지원하지 않음	✅ kubectl get nodes

04
기술 소개
“Monitoring”





04
기술 소개: Monitoring



Container Insights | Service: EKS

대시보드에 추가 | 맵에서 보기 | 성능 대시보드 보기

클러스터 상태 요약 (1)

January 6, 2025, 오후 05:47 (UTC+09:00) 기준

클러스터

eks-worker-cluster | 관련 내용 살펴보기

사용률

CPU 13% | 메모리 20%

리소스 유형별 경보 상태

클러스터	경보가 감지되지 않음
노드	경보가 감지되지 않음
네임스페이스	경보가 감지되지 않음
서비스	경보가 감지되지 않음
워크로드	경보가 감지되지 않음
포드	경보가 감지되지 않음
컨테이너	경보가 감지되지 않음

성능 및 상태 요약

지난 1분

클러스터 CPU(평균)
사용률 1%
예약됨 13%

클러스터 메모리(평균)
사용률 9%
예약됨 3%

포드(합계)
원하는 수준 30
준비 완료 13

노드(합계)
사용 불가 0
사용 가능 1

컨트롤 플레인 요약

지난 3시간

83
Max API server requests

19.1
Average API server requests latency

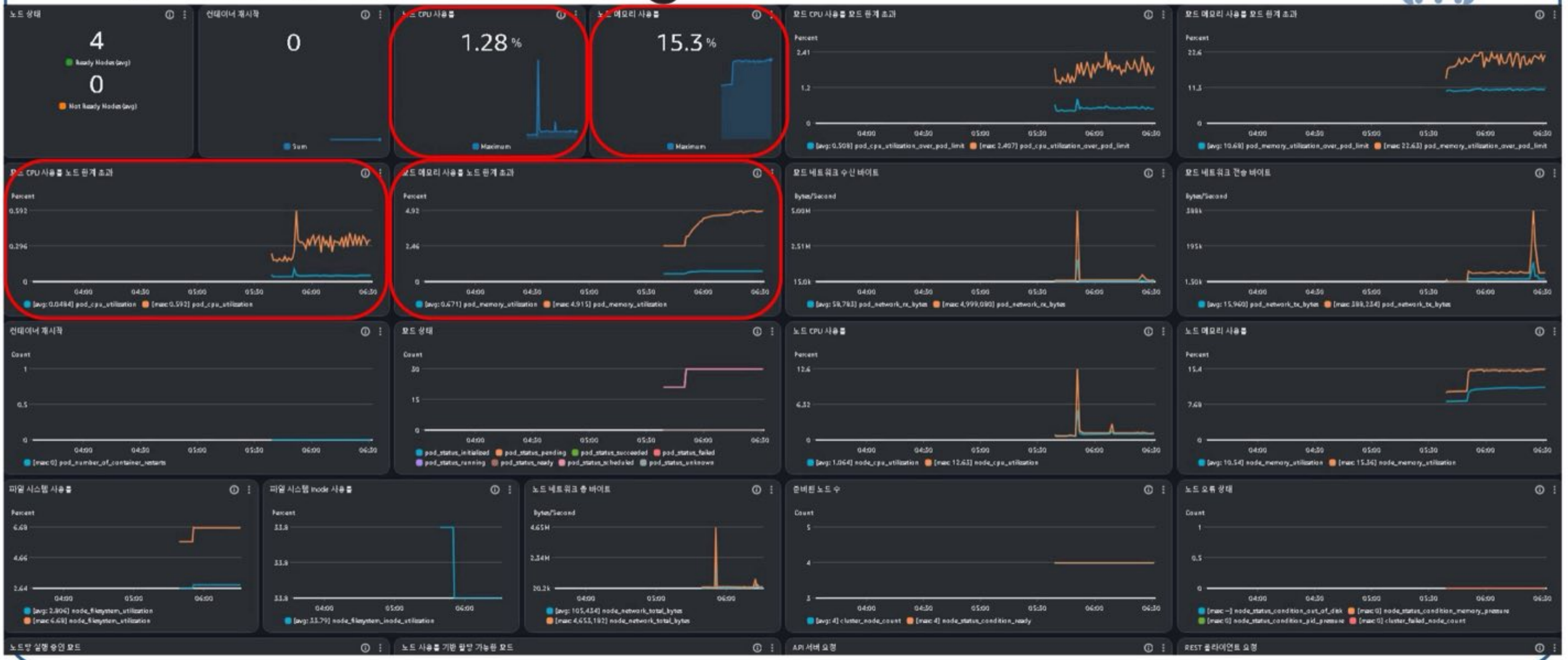
144
Total number of stored objects

0
Average admission controller latency

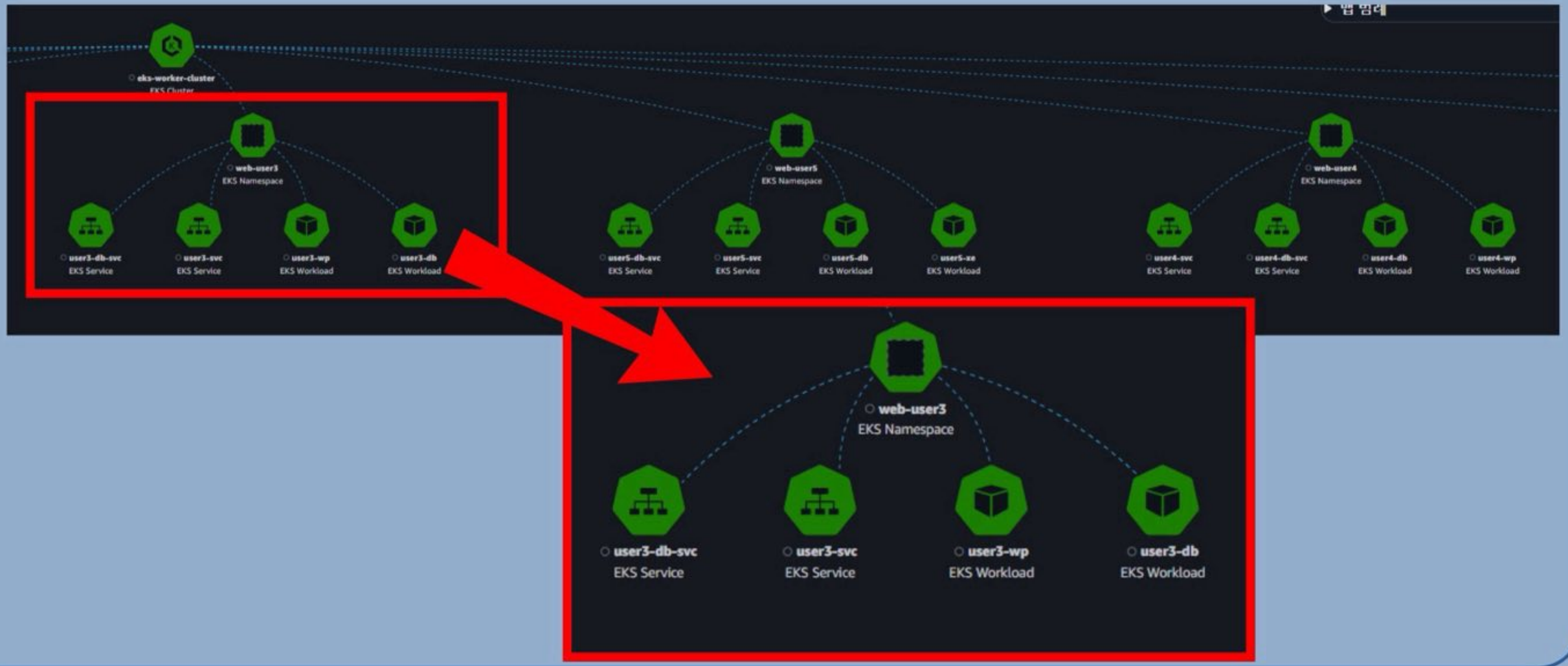
04

기술 소개: Monitoring



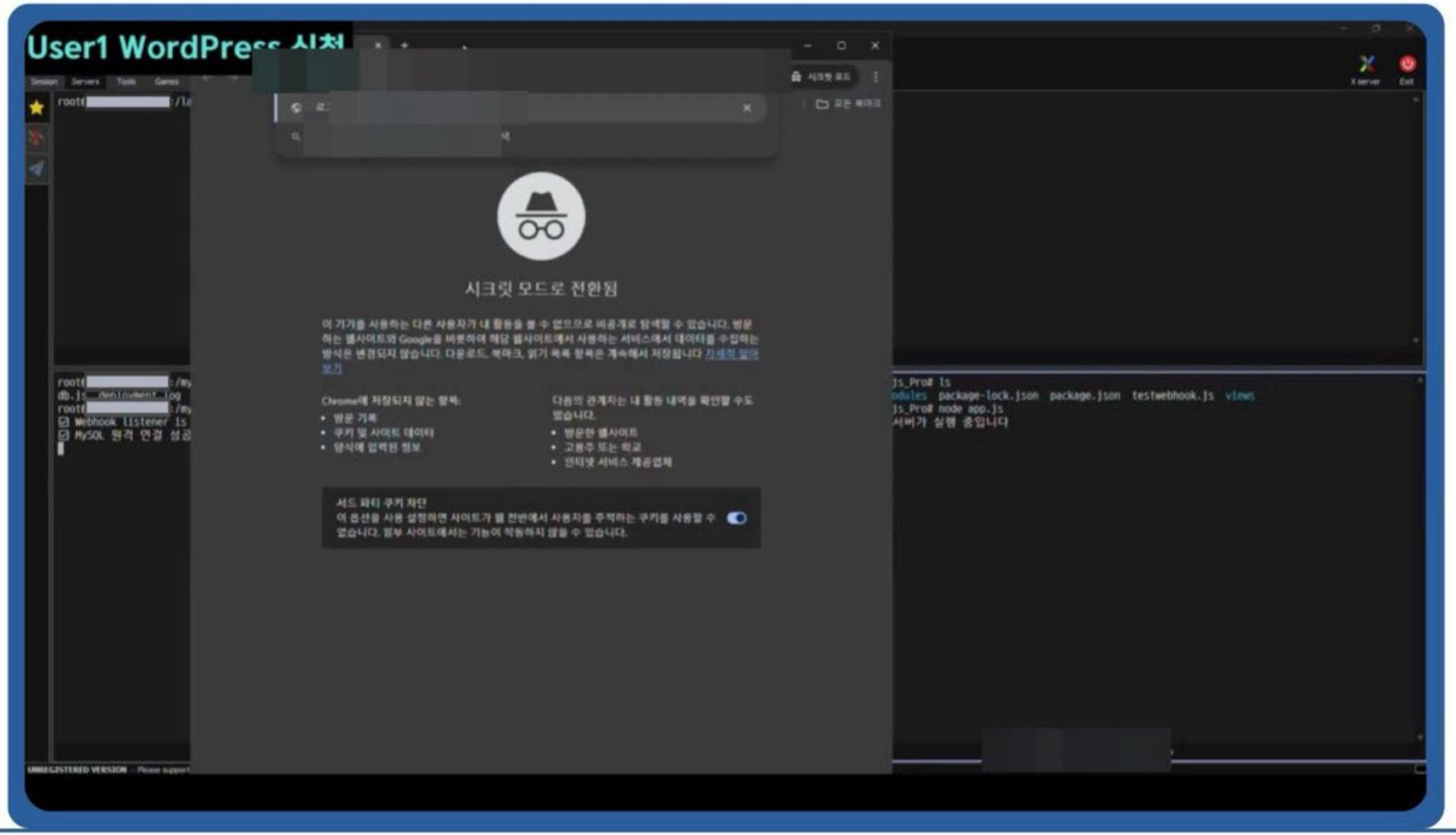


04
기술 소개: Monitoring



05

활용 방안 및 기대 효과: 시연 영상



05
활용 방안 및 기대 효과

01

배포 시간 단축

운영 비용 절감

02

사용자 맞춤형

배포 환경 제공

03

리소스 낭비

최소화 및 최적화

04

실시간 모니터링

문제 발생 시

즉시 대응

06

느낀점: 개선점

ANSIBLE

인스턴스별 관리 (Playbook)



Dynamic provisioning

06
느낀점



이번 프로젝트를 통해 Kubernetes와 자동화 도구의 편리함을 실감했으며, 복잡한 인프라를 코드로 관리하는 것이 얼마나 효율적이고 체계적인 운영을 가능하게 하는지 실감했습니다.
또한, 부족한 부분들을 개선해 나가며 더 나은 아키텍처를 설계해야겠다는 목표를 가지게 되었습니다.

Q&A

THANK YOU

2024.11.25~2024.12.10
2차 팀프로젝트

Istio 기반의 Canary Deployment와 CI/CD 파이프라인 구현



팀명: EchoCloud
팀장: 김선호
팀원 3명

LIST

1. 팀 소개
2. 프로젝트 주제 및 선정 배경
3. 개발 환경
4. 기술 소개
5. 활용방안 및 기대효과

1. Team Info

“EchoCloud”



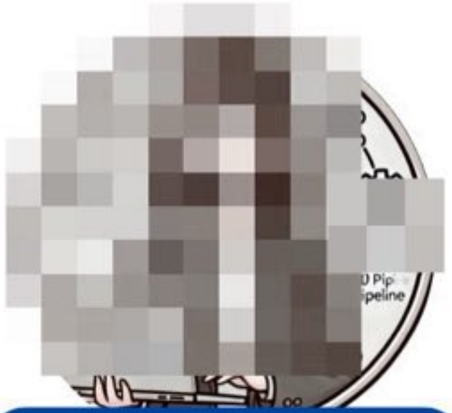
김선호

Team Leader
Istio
Multi-Master



팀원 A

Team Member
Grafana & Slack
Multi-Master



팀원 B

Team Member
CI/CD (SonarQube)
Python



팀원 D

Team Member
CI/CD (Harbor)
Python

2. 프로젝트 선정 배경

경기공유서비스
서비스 일시 중단 안내



경기공유서비스 메인화면 개편 및
서비스 안정화 작업으로 인해 서비스 접속이
일시 중단됩니다.
이용에 불편을 끼쳐드려 죄송합니다.

보다 나은 서비스 제공을 위해
항상 노력하겠습니다.

중단일시	2022년 8. 25.(목) 19시 ~ 24시
중단사유	메인화면 개편 및 서비스 안정화 작업
중단업부	경기공유서비스 전체

중단 시간은 작업 상황에 따라 변경될 수 있습니다.

매출 손실

서비스 복구
비용 증가

신뢰도 하락

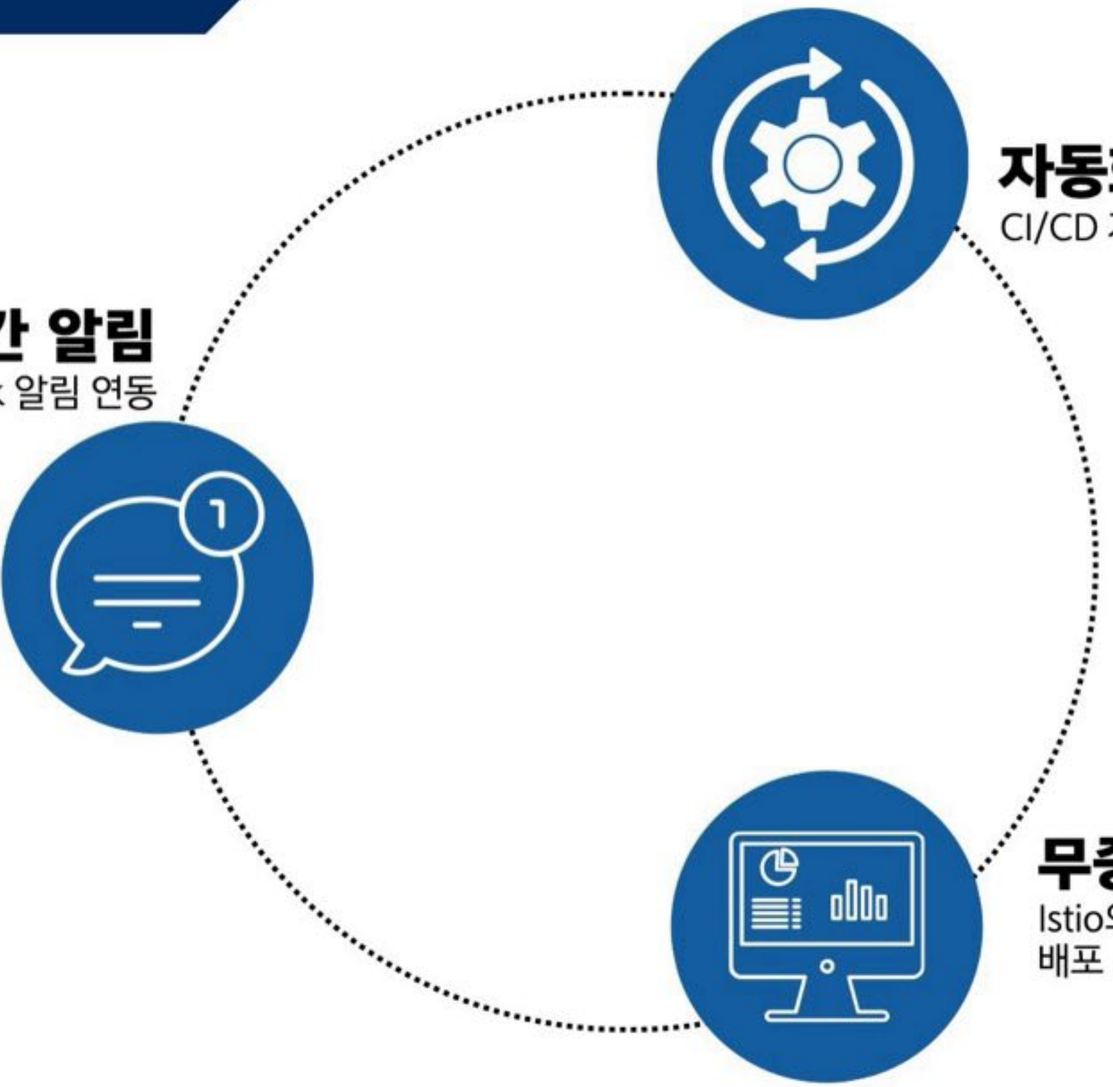
2. 프로젝트 소개

무중단 배포

[서비스 중단 없는
신뢰있고 안정적인
Canary Deployment

“환경 구축”]

2. 프로젝트 목표

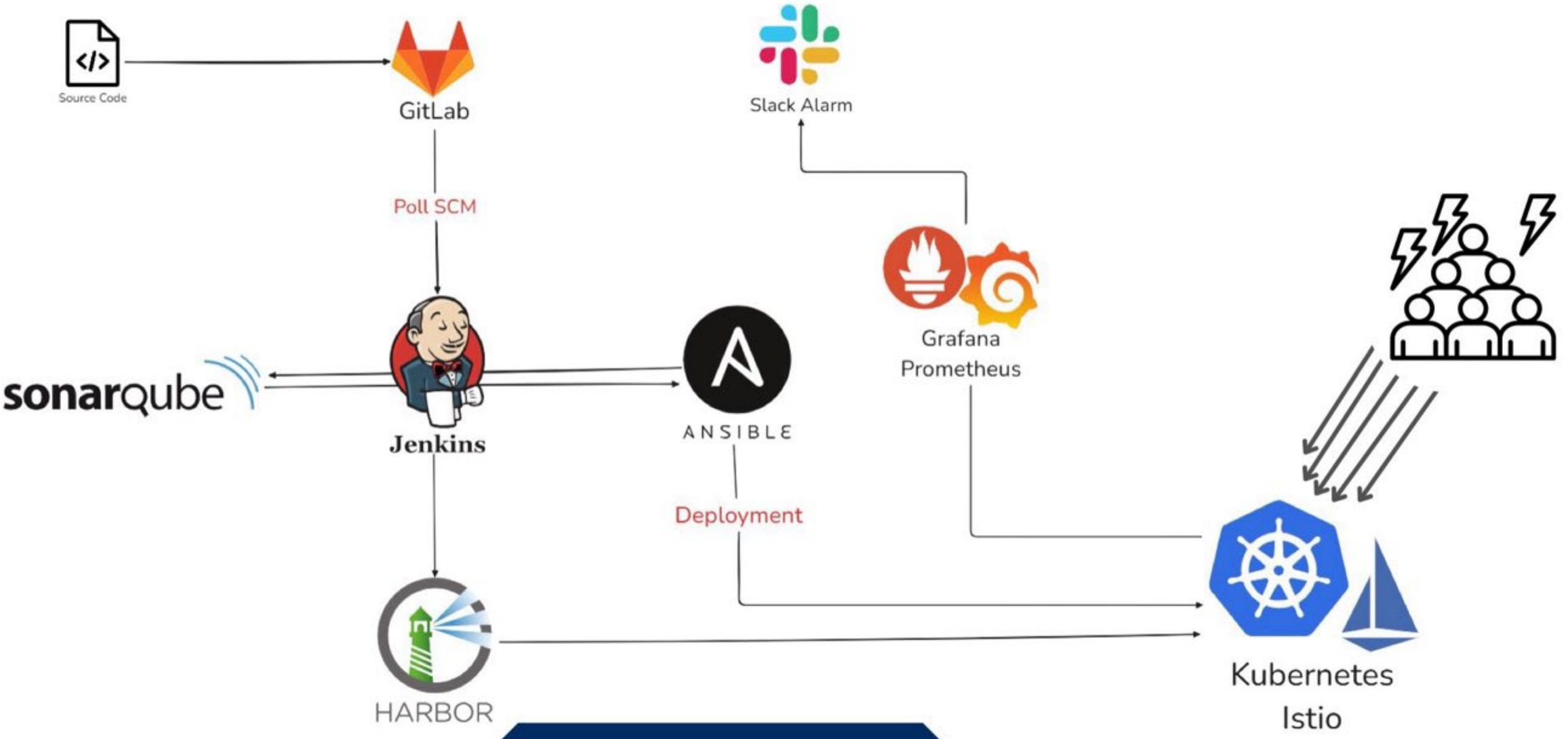


자동화
CI/CD 자동화를 통해 효율적인 배포와 관리체계 확립

실시간 알림
문제 해결 자동화 및 실시간 Slack 알림 연동

무중단 배포
Istio의 트래픽 라우팅으로 무중단배포 구현
배포 신뢰성 향상

3. Architecture



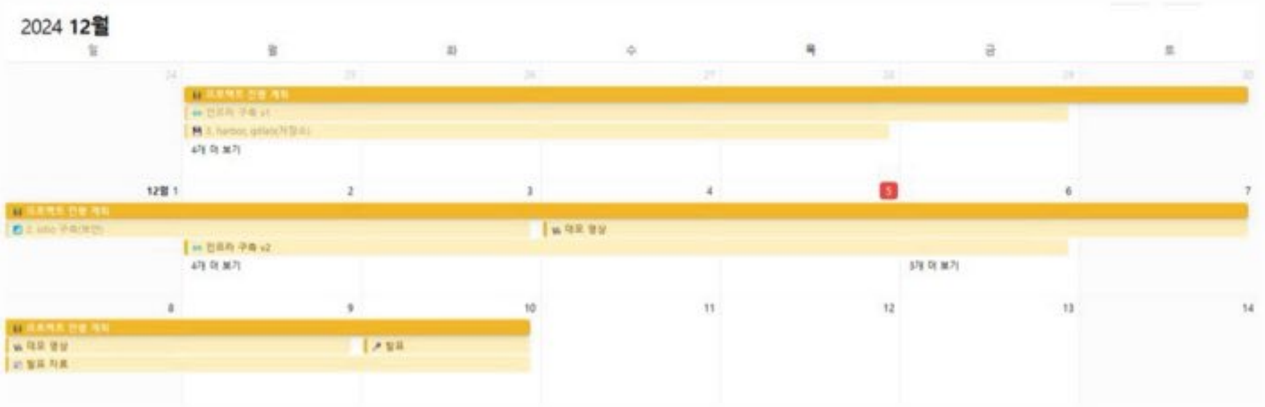
3. 개발 환경

Bridge 네트워크를 통한 효율적인 리소스 관리

추가적인 네트워크 리소스가 필요할 때 손쉽게 확장 가능

	MASTER	WORKER	JENKINS	HARBOR	GITLAB	DEV
RAM	8 GB			4 GB		
CPU	8			4		
DISK	80 GB			40 GB		
IMAGE	ubuntu 22.04					

3. 협업 도구





4. 기술 소개

Multi-Master & LoadBalancer

CI/CD Harbor/SonarQube

Istio

Grafana & Slack

4. Multi-Master

Multi-Master

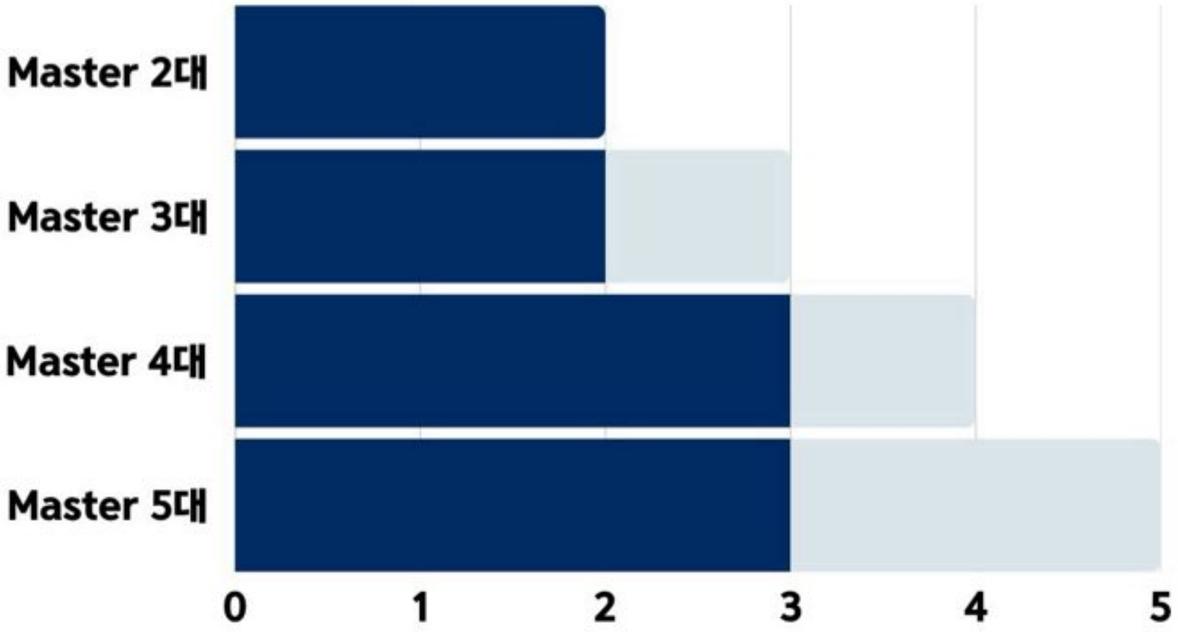
Clustser Down 방지

3대의 Master Node

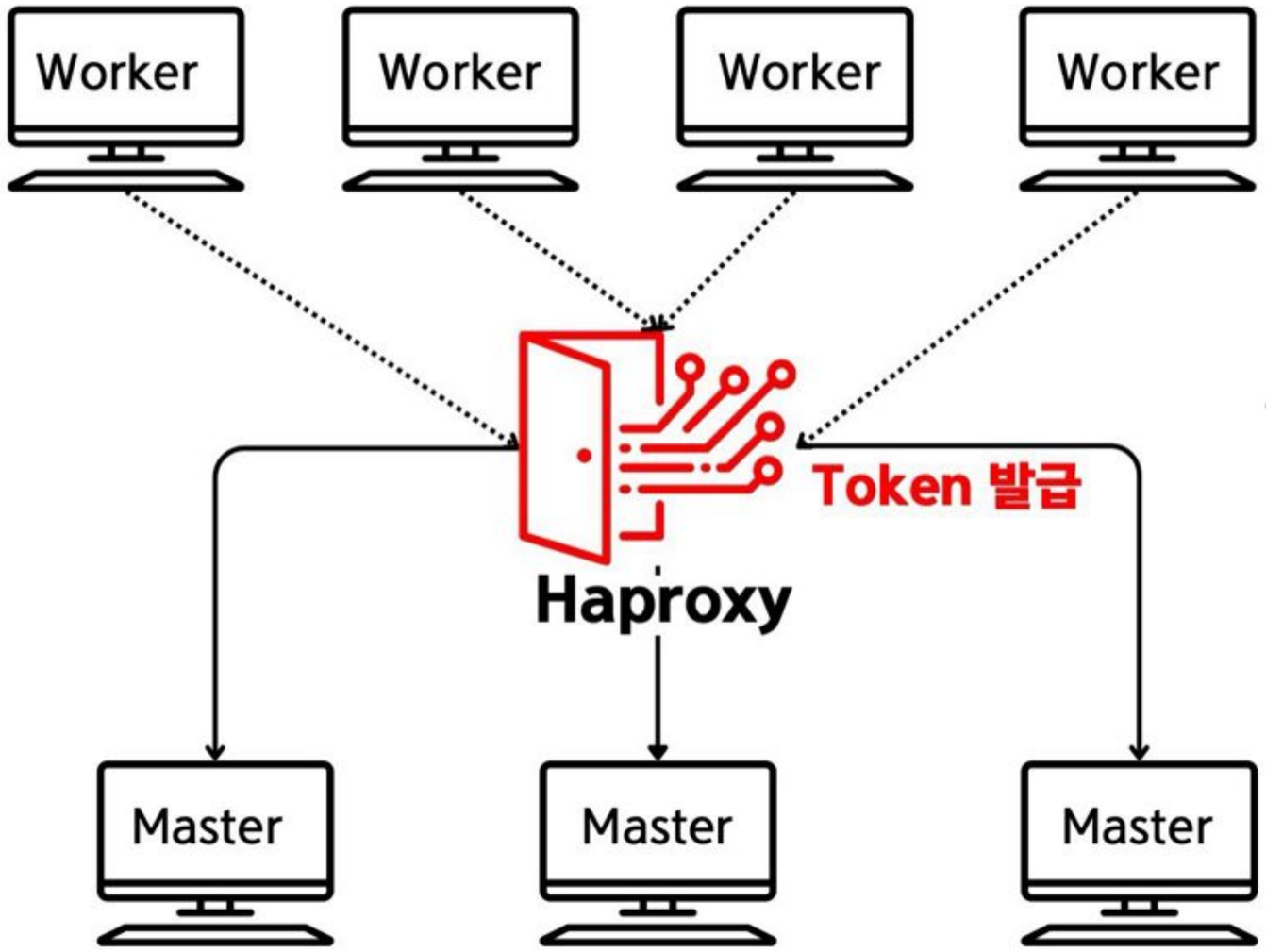
Quorum(정족수)를 고려하여 3대로 구성

Quorum(정족수)

분산 시스템에서 다수결의 원칙에 따라 의사결정을 할 때,
동의해야 하는 최소한의 노드 수



4. Load Balancer & Multi-Master



Health Check

down 된 master에게 불필요한
트래픽 전송 방지

부하분산

요청을 균등하게 분배해
클러스터 제어 성능 최적화

4. Gitlab+Jenkins, Ansible

유연한 플러그인 기반 CI 자동화 도구로,
빌드와 테스트 프로세스를 간단히 설정 가능

Jenkins

JenkinsFile

코드 저장소와 이슈 추적, 협업 기능을
통합적으로 제공하여 팀 생산성을 높임

GitLab

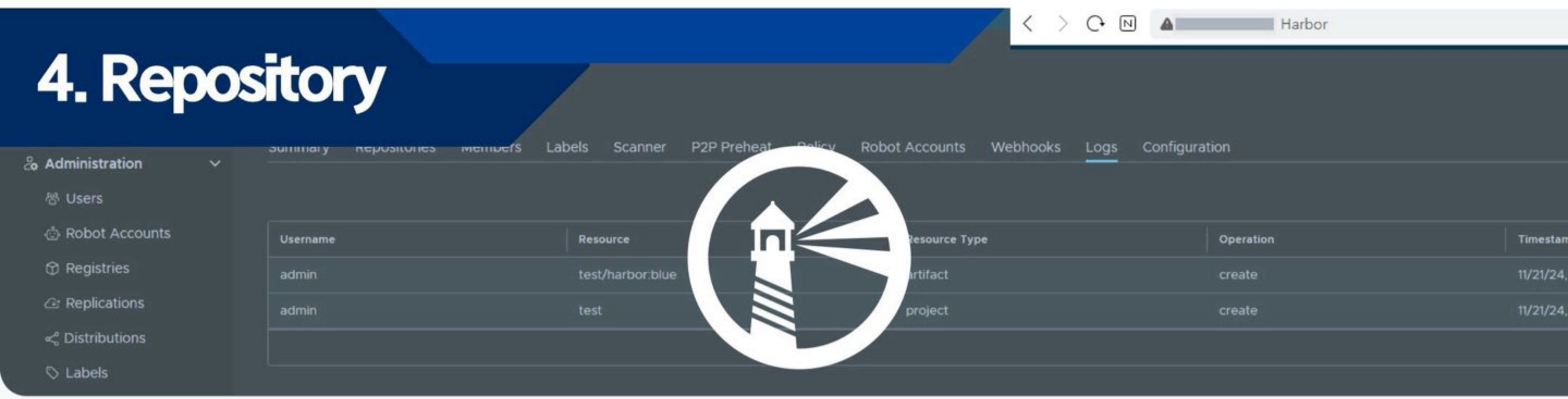


ANSIBLE

에이전트리스 방식의 간단하고 강력한 CD 도구
서버 구성 및 배포를 효율적으로 처리.

Playbook

4. Repository



Harbor

컨테이너 이미지 저장소

관리 편의

- 프로젝트별로 이미지 저장소 구분

비용 절감

- 오픈소스
- 저장 공간 및 요청 횟수에 제한 x

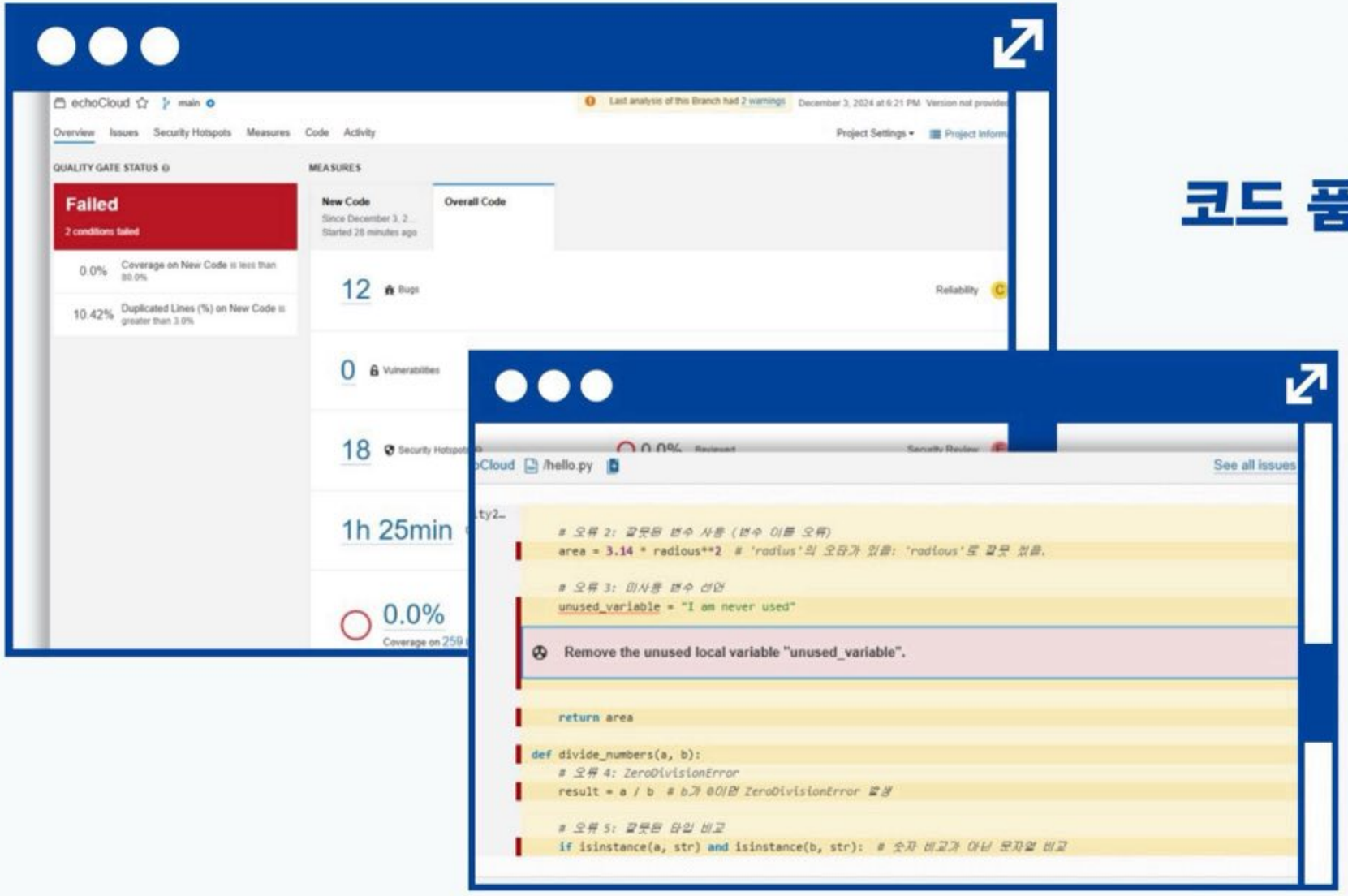
보안 강화

- CA 인증서를 지원해 데이터 암호화

RBAC

- 역할 기반 접근 제어(RBAC)로 사용자 권한을 세밀하게 관리

4. sonarQube



sonarqube

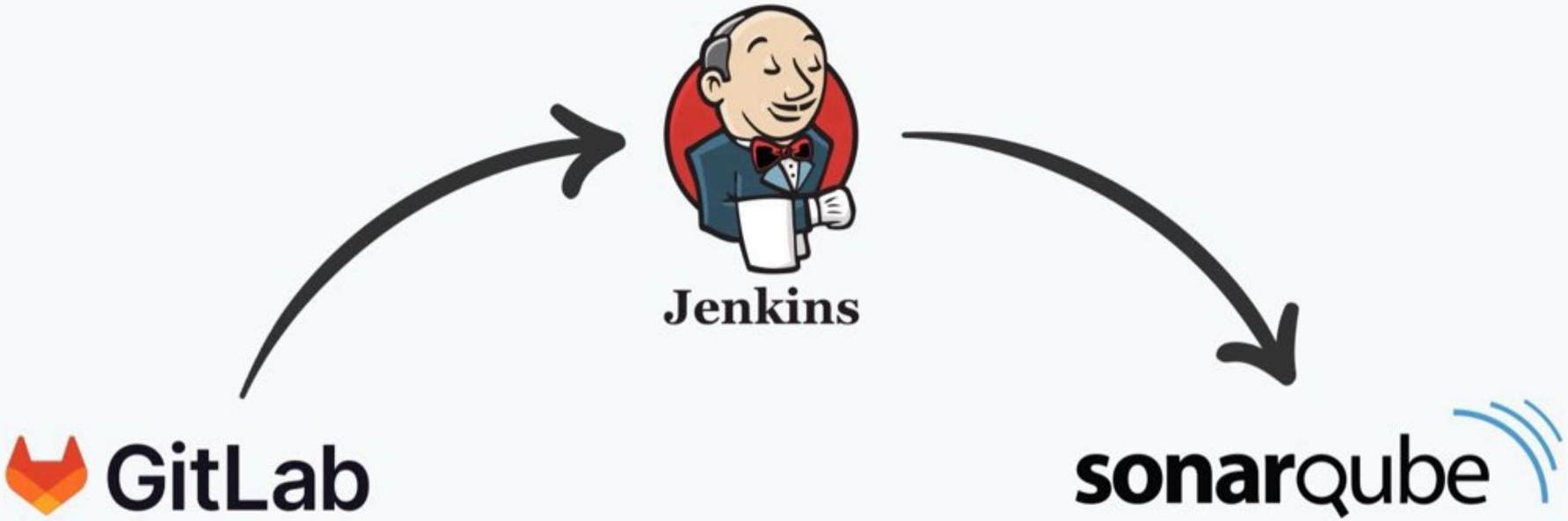
코드 품질 관리 및 정적 분석 도구

정적 코드 분석

코드 스멜

품질 게이트

4. sonarQube + Jenkins



**SonarQube는 Jenkins의 workspace를 스캔해,
Git에서 가져온 코드를 분석**

4. Python + Django



Python과 Django 프레임워크 기반 프로젝트를 SonarQube로 정적 분석하여 코드 품질 평가

4. Istio

사용 기업

Istio

Istio는 서비스 메쉬(Service Mesh)를 구현하는 오픈소스 플랫폼으로,
Kubernetes 클러스터에서 애플리케이션의 서비스 간 통신을 관리



트래픽 관리



보안



가시성



서비스 정책 대응



확장성

4. Envoy Proxy




```
root@master:~/project# k get pod -o wide
NAME                          READY   STATUS    RESTARTS   AGE
v1-dp-6fc7fb4bbd-ftqhb        2/2     Running   0          53s
v2-dp-5b9d68864d-srmsb        2/2     Running   0          53s
```

Pod 배포시 자동으로 SideCar 생성

로드 밸런싱

TLS 암호화

트래픽 라우팅

장애 복구 효율적

데이터 전송 암호화

4. CANARY UPDATE



소수 배포로 문제 발생 시 영향 최소화

리스크 최소화



일부 배포로 실시간 피드백과 문제 해결

빠른 피드백



점진적 트래픽 증가로 안정성 보장

안정성

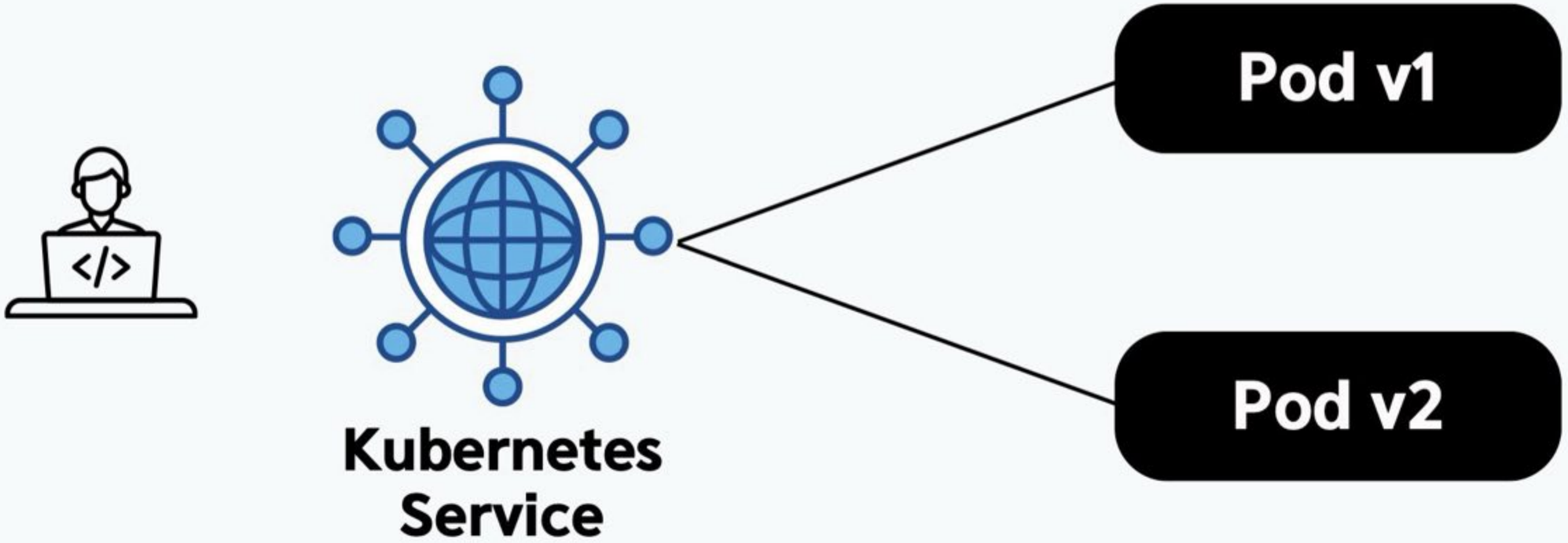


Istio와 Kubernetes로 배포 전략 구현 용이

유연성

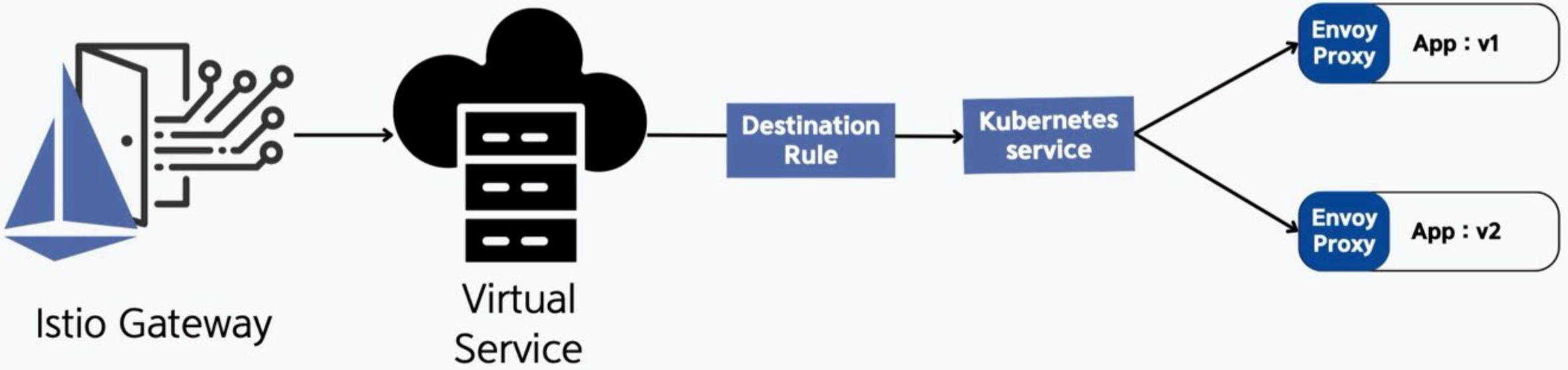
4. 기존 버전의 서비스

기존 배포



4. 새로운 버전의 서비스

Istio를 이용한 배포



4. 새로운 버전의 서비스

```yaml
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: canary-gateway
spec:
  selector:
    istio: ingressgatewa
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts:
    - "*"
```

```yaml
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: canary-virtualservice
spec:
  hosts:
  - "*"
  gateways:
  - canary-gateway
  http:
  - match:
    - headers:
        x-forwarded-for:
          exact: "[redacted]"  # 특정 IP를 v2로 라우팅
    - headers:
        x-forwarded-for:
          exact: "[redacted]"  # 특정 IP를 v2로 라우팅
    - headers:
        x-forwarded-for:
          exact: "[redacted]"  #
    route:
    - destination:
        host: canary-svc
        subset: v2
  - route:
    - destination:
        host: canary-svc
        subset: v1
```

```yaml
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: canary-destinationrule
spec:
  host: canary-svc # svc 에서 설정했던 name
  subsets:
  - name: v1
    labels:
      ver: v1
  - name: v2
    labels:
      ver: v2
```

```yaml
apiVersion: v1
kind: Service
metadata:
  name: canary-svc
  labels:
    service: canary
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v1-dp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: canary
      ver: v1
  template:
    metadata:
      labels:
        app: canary
        ver: v1
```

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: v2-dp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: canary
      ver: v2
  template:
    metadata:
      labels:
        app: canary
        ver: v2
```

```
root@Master3:~# k get svc istio-ingressgateway -n istio-system -o yaml | grep externalTrafficPolicy
  externalTrafficPolicy: Local
```

4. 새로운 버전의 서비스

Canary 배포





v1



v2 – 정상 페이지

4. Monitoring(Grafana)

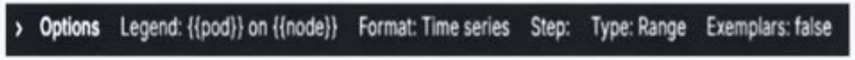




Node CPU 사용량



Pod 개수

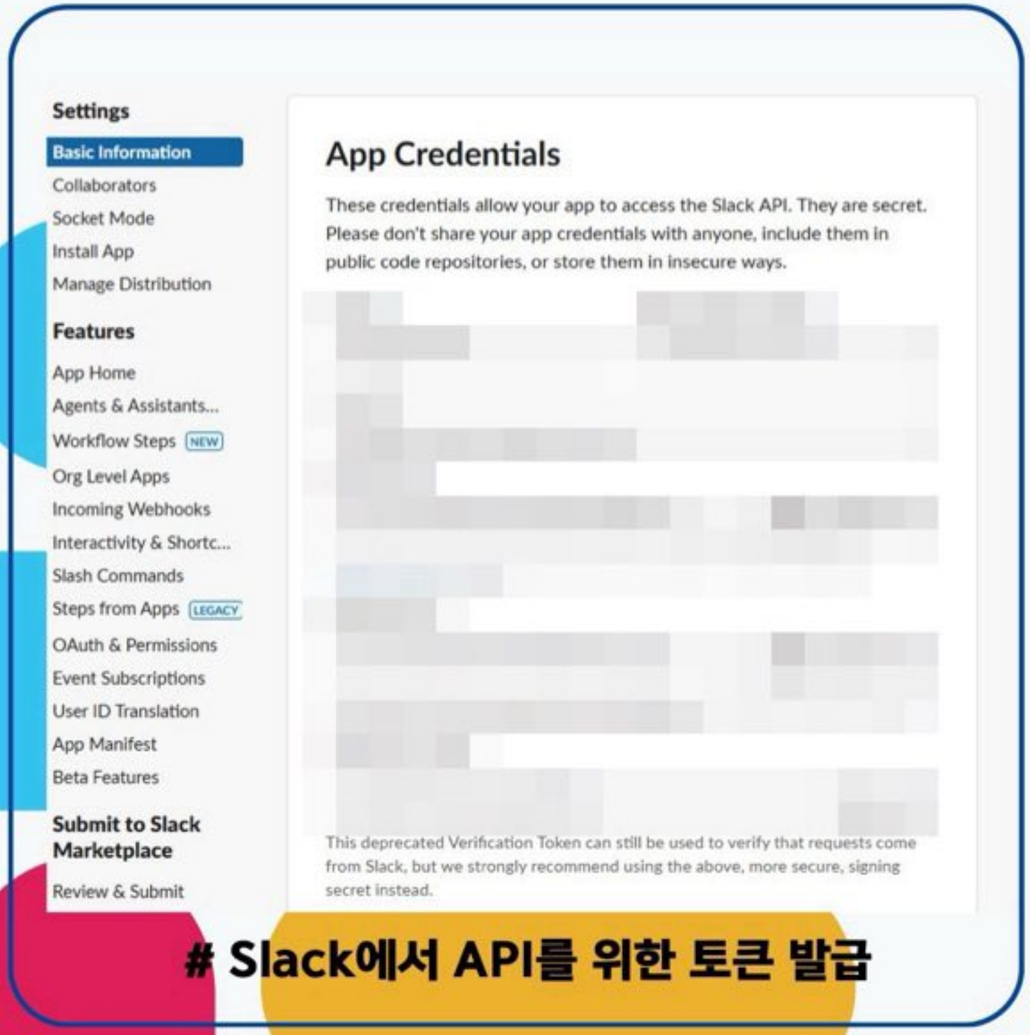


Pod CPU 사용량

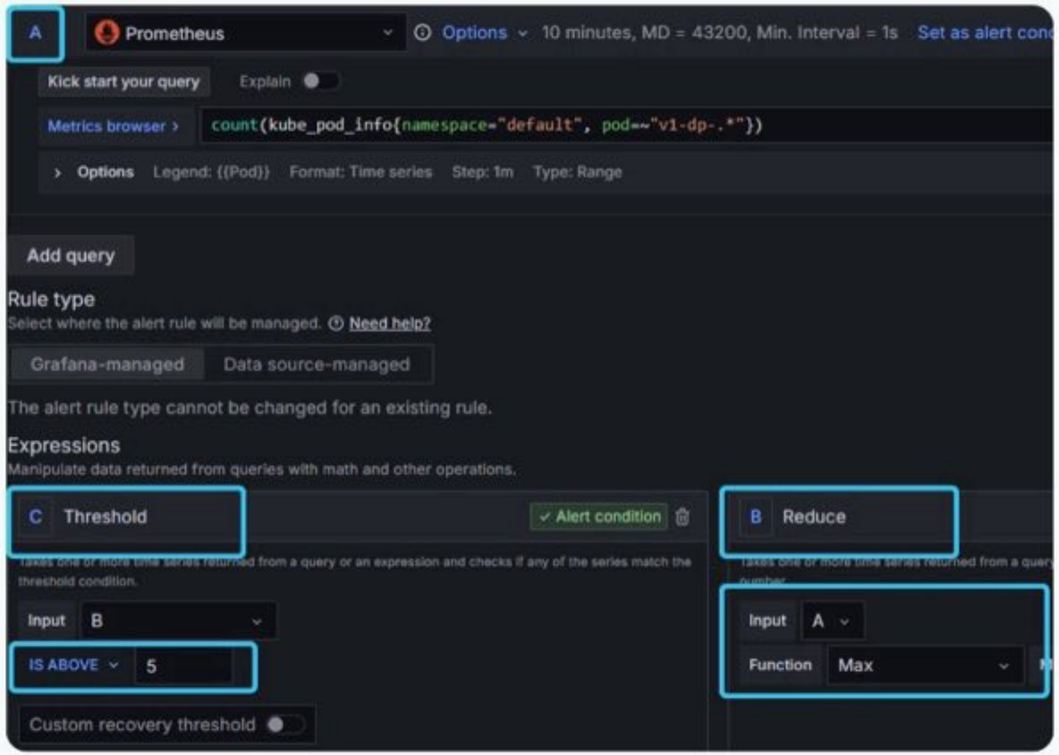


Node 메모리 사용량

4. Monitoring(Slack Alert)



Slack에서 API를 위한 토큰 발급



Grafana 에서 Alert 설정

4. Monitoring(Slack Alert)



Slack에서 alarm 확인

5. 활용 방안 및 기대 효과

보안 강화

Harbor로 배포 보안성 향상

Istio로 서비스 간 통신 보호

운영의 효율성

CI/CD 자동화로 배포 시간 단축

소나큐브를 사용해 개발자 코드 유효성 검사

배포 신뢰성 향상

Canary update로

배포의 신뢰성 향상

5. 보완점



추가하고 싶은 점

VPN

OpenVPN을 이용한
외부 네트워크 차단

Istio

Gatekeeper를 이용해
보안성 챙기기

DataBase

한 개의 Pod가 아닌
DB Pod를 두고 PVC를 통해
데이터 유지하기

Q & A

ECHO CLOUD

2024.11.25~2024.12.10 2차 팀프로젝트

Thank You

ECHO CLOUD

CI/CD 개인 프로젝트

김선호

한국정보교육원

2024. 11. 11

목차

01 기본 구성

02 k8s Cluster with kubeadm

03 Gitlab & Jenkins

04 Ansible

05 Harbor

06 Multi-Master

07 구성파일 & 작동

08 Grafana & Prometheus

01 기본 구성



Poll SCM



Jenkins

Continuous Delivery



ANSIBLE

Git Push

Docker Image Push





Deployment





Harbor

Docker Image Pull



kubernetes

01 기본 구성

Master 1,2 / Node 1,2

Memory	4 GB
Processors	4
Hard Disk (SCSI)	40 GB
CD/DVD (SATA)	Using file C:₩Us...
Network Adapter	NAT
Display	Auto detect

etc

Memory	8 GB
Processors	4
Hard Disk (SCSI)	160 GB
CD/DVD (SATA)	Using file C:₩Us...
Network Adapter	NAT
Display	Auto detect

kubernetes

Prometheus

+

Grafana

Jenkins

GitLab

Harbor

ANSIBLE

ubuntu 22.04

02 k8s Cluster with kubeadm

```
root@master:~# k get node -o wide
NAME      STATUS   ROLES           AGE     VERSION    INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION      CONTAINER-RUNTIME
master    Ready    control-plane   3d22h   v1.29.10   [redacted]    <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
master2   Ready    control-plane   43h     v1.29.10   [redacted]    <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
node1     Ready    <none>          3d22h   v1.29.10   [redacted]    <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
node2     Ready    <none>          3d22h   v1.29.10   [redacted]    <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
```

- kubeadm을 이용하여 k8s Cluster을 구성

```
root@master:~# k get pod,ipaddresspool -n metallb-system -o wide
NAME                                READY   STATUS    RESTARTS       AGE   IP            NODE      NOMINATED NODE   READINESS GATES
pod/controller-5c6b6c8447-4gd8f     1/1     Running   4 (43m ago)    43h   [redacted]    node2     <none>           <none>
pod/speaker-2wb5l                   1/1     Running   7 (33m ago)    43h   [redacted]    master2   <none>           <none>
pod/speaker-bxw5k                   1/1     Running   6 (33m ago)    43h   [redacted]    master    <none>           <none>
pod/speaker-rktgc                   1/1     Running   8 (32m ago)    43h   [redacted]    node2     <none>           <none>
pod/speaker-v449p                   1/1     Running   7 (33m ago)    43h   [redacted]    node1     <none>           <none>

NAME                                AUTO ASSIGN   AVOID BUGGY IPS   ADDRESSES
ipaddresspool.metallb.io/doro-pool  true          false             ["[redacted]"]
```

- metallb을 이용하여 LoadBalancing svc 구현

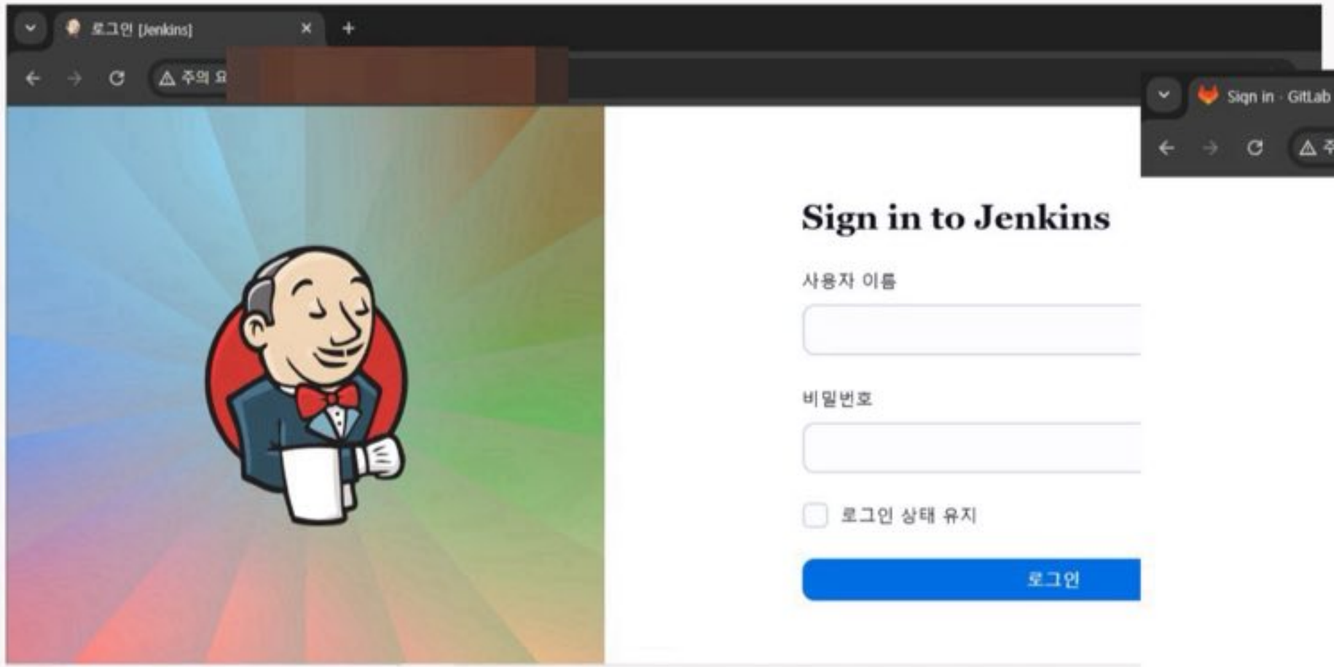
```
root@master:~# k get L2Advertisement -n metallb-system
NAME      IPADDRESSPOOLS   IPADDRESSPOOL SELECTORS   INTERFACES
doro-l2   ["doro-pool"]
```

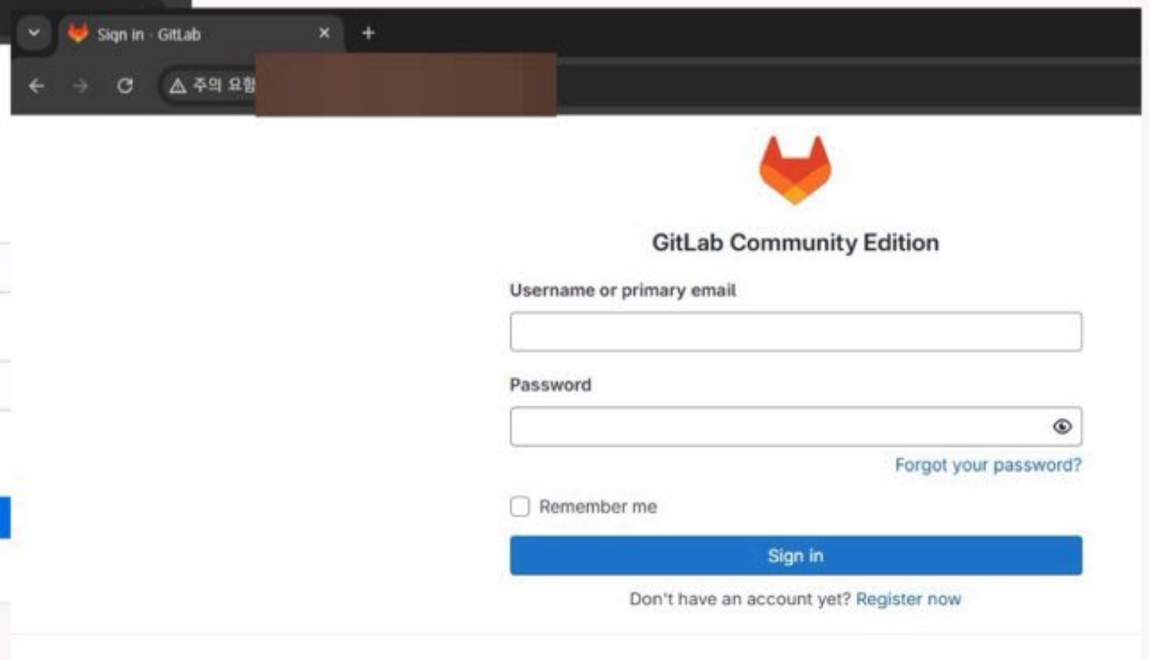
- 추가적으로 L2 Net 생성

03 Gitlab & Jenkins install

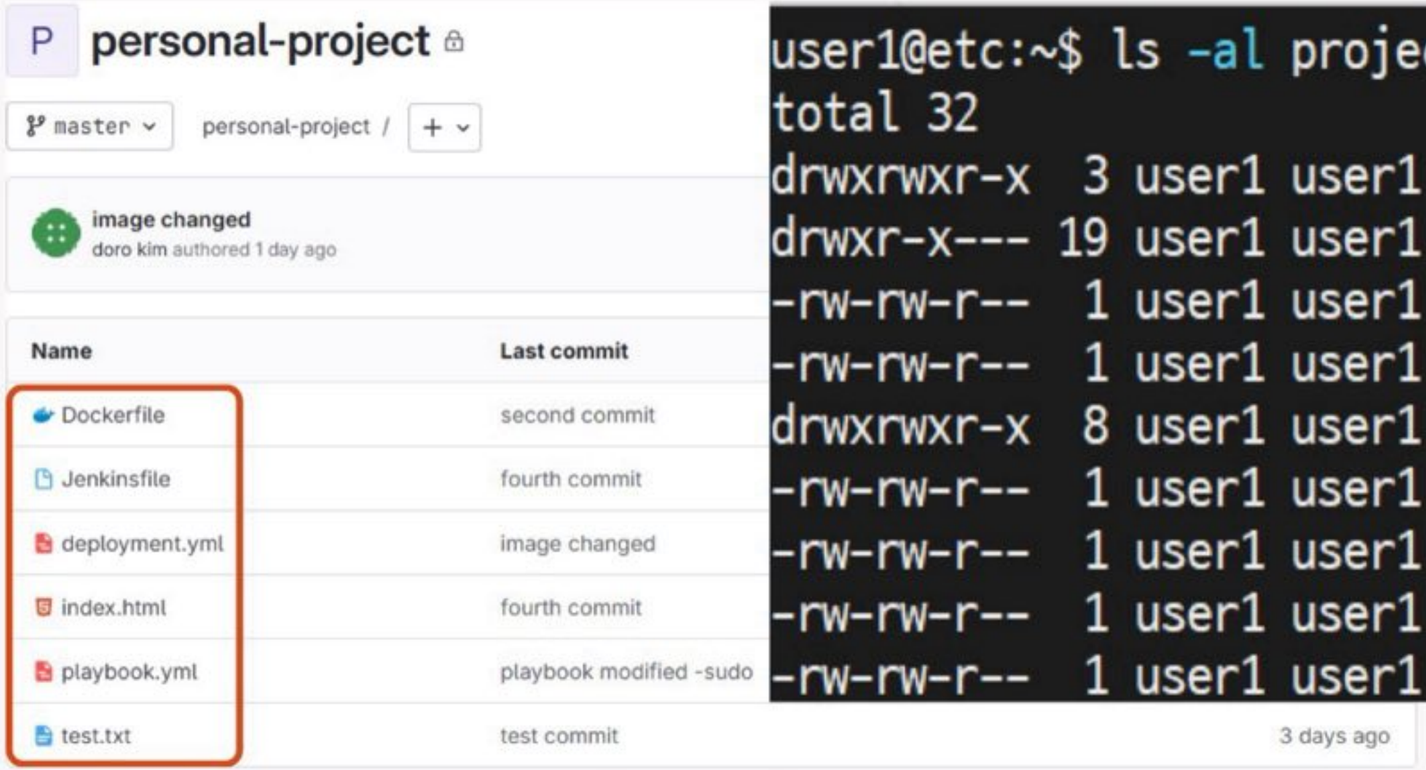
```
root@etc:~# gren '^external_url' /etc/gitlab/gitlab.rb
external_url
root@etc:~# grep '^HTTP_PORT' /etc/default/jenkins
HTTP_PORT=8090
```

- Gitlab의 포트번호를 8008, Jenkins의 포트번호는 8090으로 변경





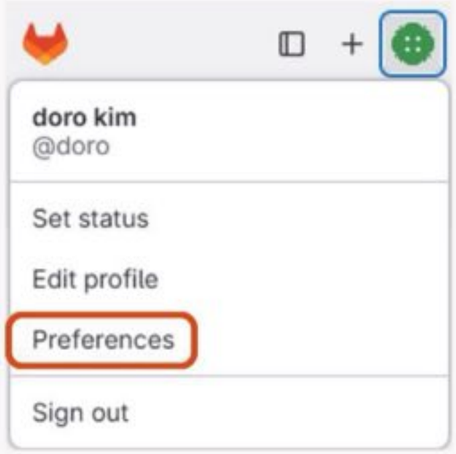
03 Gitlab & Jenkins in Gitlab

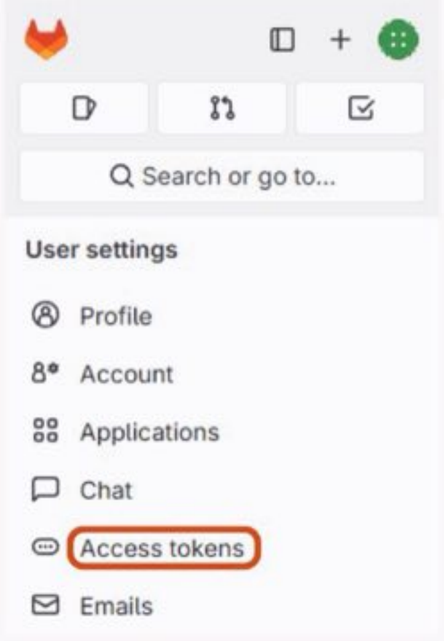



```
user1@etc:~$ ls -al project/
total 32
drwxrwxr-x  3 user1 user1 4096 11월  7 00:13 .
drwxr-x--- 19 user1 user1 4096 11월  9 20:29 ..
-rw-rw-r--  1 user1 user1  968 11월  9 20:24 deployment.yml
-rw-rw-r--  1 user1 user1   58 11월  9 00:57 Dockerfile
drwxrwxr-x  8 user1 user1 4096 11월  9 20:29 .git
-rw-rw-r--  1 user1 user1   18 11월  9 02:14 index.html
-rw-rw-r--  1 user1 user1  599 11월  9 02:15 Jenkinsfile
-rw-rw-r--  1 user1 user1  605 11월  9 20:19 playbook.yml
-rw-rw-r--  1 user1 user1    0 11월  6 23:56 test.txt
```

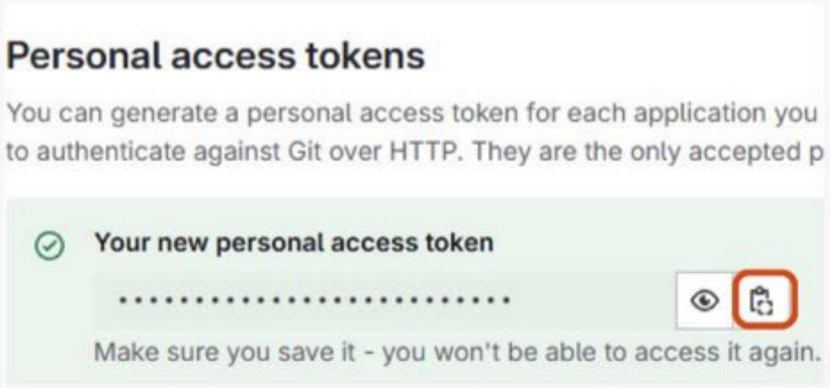
- git lab에서 root의 권한으로 doro 사용자 & Project 생성
- etc node 의 user1 사용자에서 project 라는 디렉토리와 gitlab의 personal-project와 연결

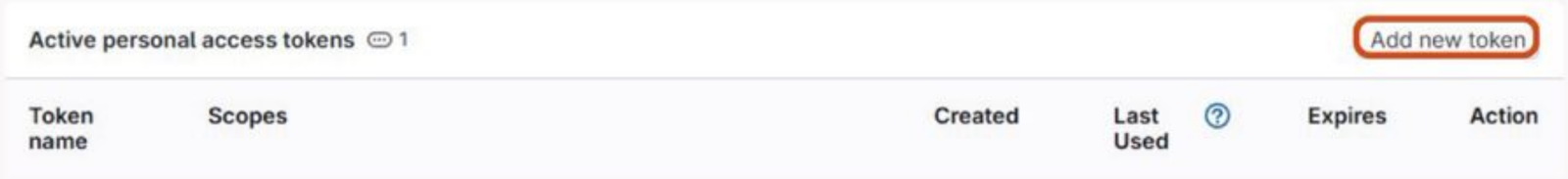
03 Gitlab & Jenkins in Gitlab





- Jenkins에게 인증하기 위한 Token 값
- 이전에 생성하였던 doro user로 로그인
- Token 생성 후 복사





03 Gitlab & Jenkins in Jenkins



New credentials

Kind

Username with password

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

Gitlab user 명 (doro)

Treat username as secret ?

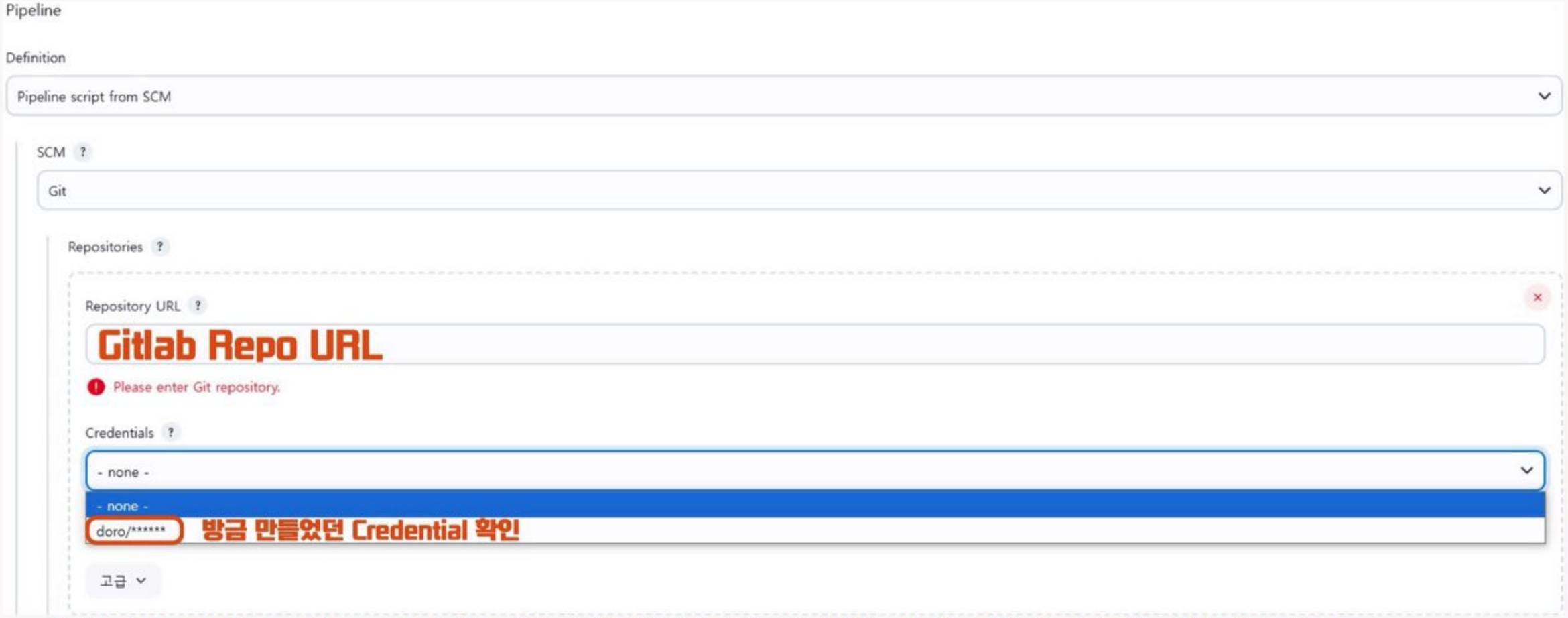
Password ?

복사한 토큰 값

ID ?

Credential 이름 (추후에 Jenkins file에서 사용함)

03 Gitlab & Jenkins in Jenkins



- 새로운 Pipeline 구성 시 이렇게 설정 해주면 Gitlab과 연결 가능

03 Gitlab & Jenkins in Jenkins

Build Triggers

- [ ] Build after other projects are built ?
- [ ] Build periodically ?
- [ ] Build when a change is pushed to GitLab. GitLab webhook URL [redacted]
- [ ] GitHub hook trigger for GITScm polling ?
- [x] Poll SCM ?

Schedule ?

H/5 * * * * **매 5분마다 실행하게 설정**

Would last have run at Monday, November 11, 2024, 1:21:01 AM Korean Standard Time; would next run at Monday, November 11, 2024, 1:26:01 AM Korean Standard Time.

03 Gitlab & Jenkins settings

```
user1@etc:~$ cat /etc/passwd ¦ grep jenkins
jenkins:x:130:137:Jenkins,,,:/var/lib/jenkins:/bin/bash
user1@etc:~$ sudo ls -al /var/lib/jenkins/.ssh/
total 24
drwxr-xr-x  2 jenkins jenkins 4096 11월  9 01:31 .
drwxr-xr-x 18 jenkins jenkins 4096 11월 10 21:08 ..
-rw-r--r--  1 jenkins jenkins  109 11월  9 01:30 config
-rw-------  1 jenkins jenkins 2934 11월  9 01:31 known_hosts
-rw-------  1 jenkins jenkins 2098 11월  9 01:31 known_hosts.old
-rw-------  1 jenkins jenkins 2590 11월  9 01:13 projectkey.pem
```

- Ansible을 위한 .ssh 디렉토리 아래에 Key생성 & Config 파일 구성
- Public Key는 복사하여 각 노드의 user1/.ssh/authorized_keys에 복사

```
user1@etc:~$ sudo cat /etc/sudoers ¦ grep jenkins
jenkins ALL=(ALL)       NOPASSWD: ALL
user1@etc:~$ groups jenkins
jenkins : jenkins docker
user1@etc:~$
```

- 비밀번호 없이 sudo 권한을 이용하고, docker 사용을 위해 group 추가

04 Ansible playbook

```
user1@etc:~/project$ cat playbook.yml
---
- name: Playbook 과 Jenkins를 이용한 CICD
  hosts: master
  become: true
  tasks:

    - name: Master에게 yml file 복사
      copy:
        src: deployment.yml
        dest: /home/user1/project_jenkins/deployment.yml
        owner: user1
        group: user1
        mode: '0664'

    - name: deployment 배포
      shell: sudo kubectl apply -f /home/user1/project_jenkins/deployment.yml

    - name: Services 목록 출력
      shell: sudo kubectl get svc
      register: svc_output

    - name: 배포한 Deployment 공인 IP 확인
      debug:
        var: svc_output.stdout
```

/etc/ansible/hosts에 master를 [redacted] 으로 등록

sudo 권한으로 실행하도록 설정

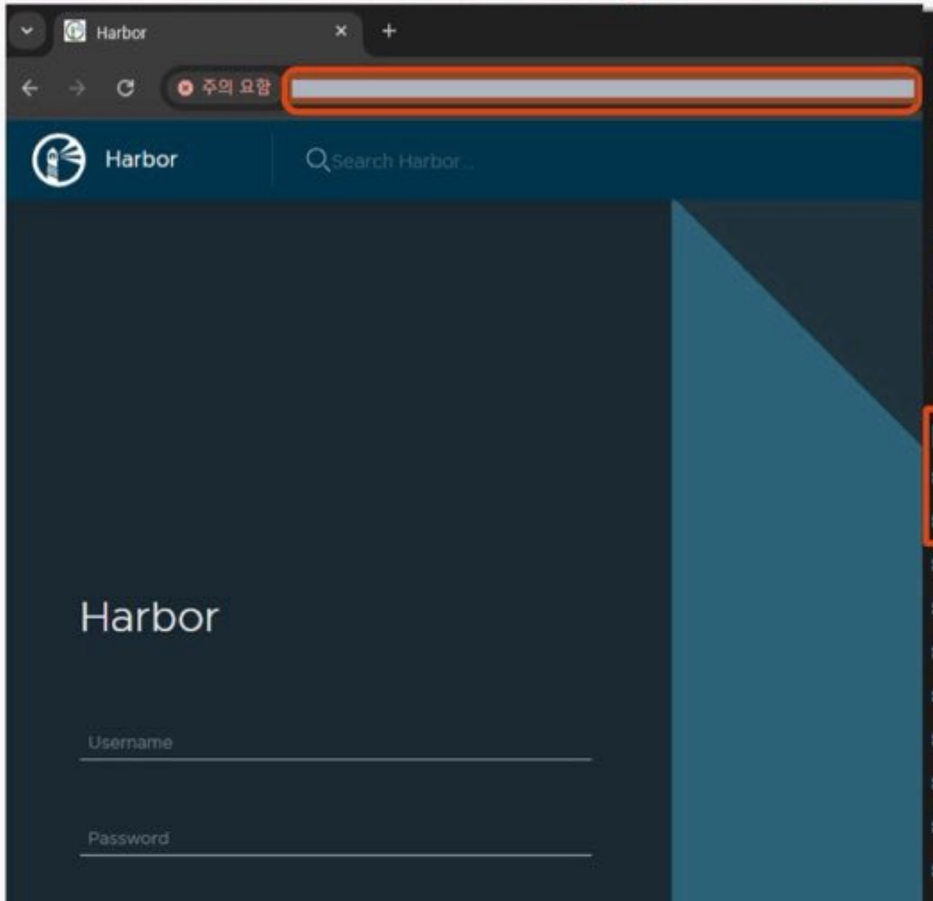
Jenkins가 가지고 있는 deployment.yml 을 master1 에게 복사

복사하여 user1/project_jenkins 아래에 있는 deployment 파일을 apply

Jenkins Console 창에서 공인 IP 확인 할 수 있게 output 설정

05 Harbor

- Harbor를 사용하여 컨테이너 이미지를 효율적으로 관리하고 접근을 제어
- HTTPS 인증서를 설정하여 전송 데이터의 암호화 & 외부 공격에 대한 보안 강화
- 역할 기반 접근 제어(RBAC)로 사용자 권한을 세밀하게 관리

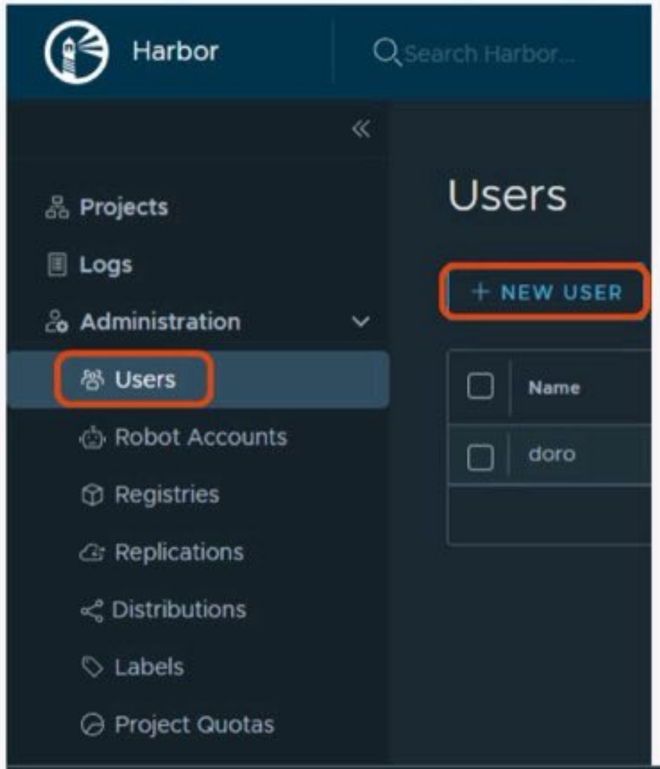


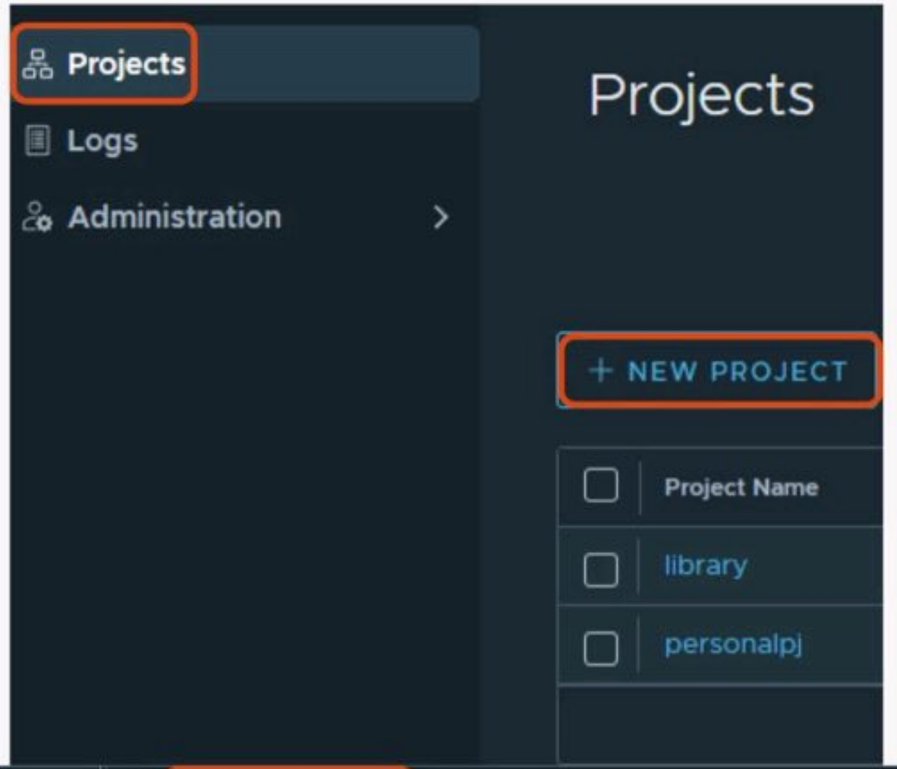

```
root@etc:~/harbor# cat /etc/docker/daemon.json
{ 모든 Node & Master에게 내용추가
   "insecure-registries" : ["                    "]
}
root@etc:~/harbor#
```

https 접속 확인

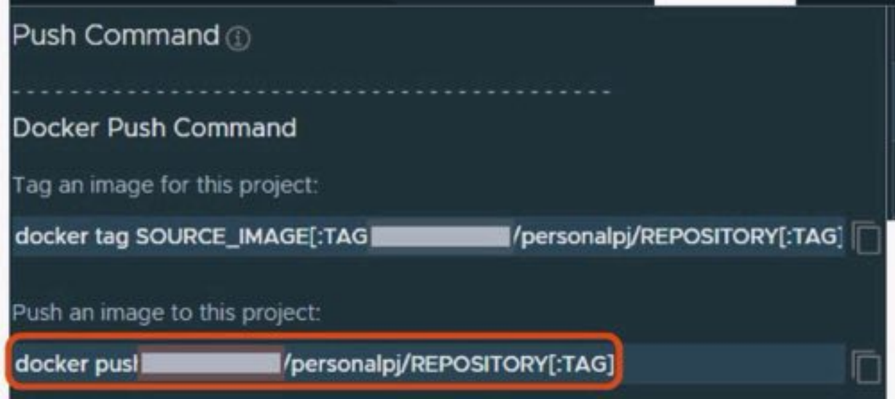
```
root@etc:~/harbor# curl -v                    --insecure
*   Trying                .
* Connected to                (              ) port 443 (#0)
* ALPN, offering h2
* ALPN, offering http/1.1
* TLSv1.0 (OUT), TLS header, Certificate Status (22):
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.2 (IN), TLS header, Certificate Status (22):
* TLSv1.2 (IN), TLS handshake, Certificate (11):
```

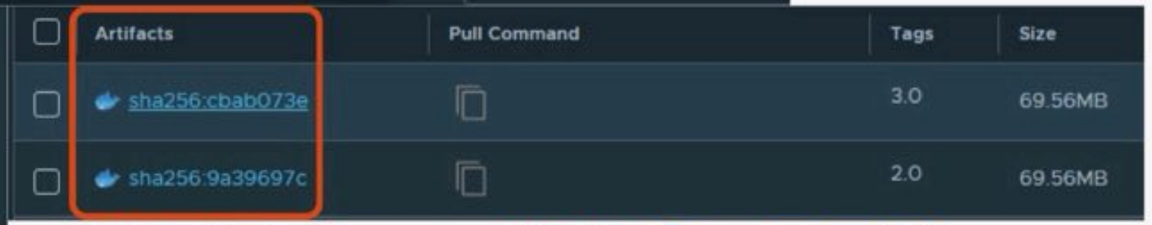
05 Harbor











- 새로운 User 생성 후, 로그인
- Project 생성 후 PUSH COMMAND 확인

06 Multi-Master with etcd

- LoadBalancer를 사용하면 마스터 노드 간 트래픽을 자동으로 분산하여, 단일 노드 장애 시에도 클러스터 관리 기능이 지속 가능하다는 큰 장점
- 리소스 한계로 인해 k8s 멀티 마스터 환경을 LoadBalancer 없이 구성하여 두 개의 마스터 노드를 클러스터로 묶어 고가용성을 확보

```
root@master:~# k get node -o wide
NAME      STATUS   ROLES           AGE    VERSION    INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION      CONTAINER-RUNTIME
master    Ready    control-plane   3d22h  v1.29.10                 <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
master2   Ready    control-plane   43h    v1.29.10                 <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
node1     Ready    <none>          3d22h  v1.29.10                 <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
node2     Ready    <none>          3d22h  v1.29.10                 <none>        Ubuntu 22.04.5 LTS   6.8.0-48-generic    docker://27.3.1
```

```
root@Master2:~# etcdctl member list --write-out=table
+------------------+---------+---------+----------------------------+----------------------------+
|        ID        | STATUS  |  NAME   |         PEER ADDRS         |        CLIENT ADDRS        |
+------------------+---------+---------+----------------------------+----------------------------+
| 5dd5783069d597cd | started |  master |                            |                            |
| ce4060ccf04b2727 | started | master2 |                            |                            |
+------------------+---------+---------+----------------------------+----------------------------+
```

- 정상적으로 etcd 까지 동기화 되어 있는 모습

06 Multi-Master with etcd save snapshot

```
root@master:~# etcdctl snapshot save /tmp/etcd-backup
2024-11-17 16:38:34.009811 I | clientv3: opened snapshot stream; downloading
2024-11-17 16:38:34.068635 I | clientv3: completed snapshot read; closing
Snapshot saved at /tmp/etcd-backup
root@master:~# ls -al /tmp/
total 6208
drwxrwxrwt 15 root root    4096 11월 17 16:38 .
drwxr-xr-x 20 root root    4096 11월  6 22:27 ..
-rw-r--r--  1 root root 6291488 11월 17 16:38 etcd-backup
```

- etcd snapshot 저장 완료

```
root@master:~# k get pod,deployment
NAME                              READY   STATUS    RESTARTS      AGE
pod/doro-dp-bc44f5559-5jgwj       1/1     Running   1 (40h ago)   40h
pod/doro-dp-bc44f5559-rfbn2       1/1     Running   1 (40h ago)   40h

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/doro-dp      2/2     2            2           40h
```

→

```
root@master:~# k delete deployment doro-dp
deployment.apps "doro-dp" deleted
root@master:~# k get pod,deployment
No resources found in default namespace.
```

- etcd 복구 테스트를 위해 deployment를 삭제

06 Multi-Master with etcd restore

```
root@master:~# ETCDCTL_API=3 etcdctl --data-dir /var/lib/etcd-new snapshot restore /tmp/etcd-backup
2024-11-17 16:50:17.421436 I | mvcc: restore compact to 28169
2024-11-17 16:50:17.437038 I | etcdserver/membership: added member 8e9e05c[redacted]d [redacted] to cluster cdf818194e3a8c32
root@master:~# tree /var/lib/etcd-new
/var/lib/etcd-new
└── member
    ├── snap
    │   ├── [redacted].snap
    │   └── db
    └── wal
        └── [redacted].wal

3 directories, 3 files
```

- 스냅샷을 이용하여 복구

```
  - hostPath:
      path: /var/lib/etcd-new
      type: DirectoryOrCreate
    name: etcd-data
```

- /etc/kubernetes/manifests/etcd.yaml 파일의 하단에 있는 /var/lib/etcd -> /var/lib/etcd-new로 변경

06 Multi-Master with etcd restore

```
root@master:~# k get pod,deploy & date
[1] 35818
2024. 11. 17. (일) 16:59:15 KST
root@master:~# NAME                                READY   STATUS    RESTARTS       AGE
pod/doro-dp-bc44f5559-5jgwj   1/1     Running   1 (40h ago)   41h
pod/doro-dp-bc44f5559-rfbn2   1/1     Running   1 (40h ago)   41h

NAME                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/doro-dp   2/2     2            2           41h
```

- 정상적으로 스냅샷을 불러와서 pod가 복구 된 모습

07 구성파일 & 작동 Jenkinsfile

```
user1@etc:~/project$ cat Jenkinsfile
pipeline {
  agent any
  stages {
    stage('gitlab scm update') {
      steps {
        git url:                                                    branch: 'master', credentialsId: 'doro-token'
      }
    }
    stage('이미지생성') {
      steps {
        sh '''
        sudo docker build -t              /personalpj/doro:3.0 .
        sudo docker push                  /personalpj/doro:3.0
        '''
      }
    }
    stage('Ansible을 이용해서 Master 에게 Deployment 배포 명령') {
      steps {
        sh '''
        ansible-playbook playbook.yml
        '''
      }
    }
  }
}
```

gitlab과 연동하는데 필요한 url과 credentialsId 작성

이미지를 만들어 Harbor로 Push

playbook을 실행

07 구성파일 & 작동 deployment file

```
user1@etc:~/project$ cat deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: doro-dp
spec:
  replicas: 3
  selector:
    matchLabels:
      rabbit: doro
  template:
    metadata:
      labels:
        rabbit: doro
    spec:
      containers:
      - name: doro-ctn
        image: [redacted]/personalpj/doro:3.0
        resources:
          limits:
            memory: "64Mi"
            cpu: "250m"
        ports:
        - containerPort: 80
      imagePullSecrets:
        - name: dockerauth
---
apiVersion: v1
kind: Service
metadata:
  name: doro-svc
spec:
  selector:
    rabbit: doro
  ports:
  - port: 80
    targetPort: 80
  type: LoadBalancer
---
apiVersion: v1
kind: Secret
metadata:
  name: dockerauth
type: kubernetes.io/dockerconfigjson
data:
  .dockerconfigjson: --------
```

image를 harbor에서 당겨옴

노드에서 docker login 할 필요 없이 secret 으로 docker 정보를 기입

기존에 구성하였던 metallb로 LoadBalancer svc 사용

--------〉 본인의 .docker/config 를 base64로 암호화 하여 그 값을 입력

07 구성파일 & 작동 in Jenkins

```
root@Master2:~# k get pods,svc,secret
NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   [redacted]   <none>        443/TCP   4d2h
```

Today

- #17 오전 1:28
- #16 오전 1:25
- #15 오전 1:24

```
+ sudo docker push [redacted]/personalpj/doro:3.0
The push refers to repository [[redacted]/personalpj/doro]
7a6b7bb75c0e: Preparing
e4e9e9ad93c2: Preparing
6a[redacted]: Preparing
8ce189049cb5: Preparing
296af1bd2844: Preparing
63d7ce983cd5: Preparing
b33db0c3c3a8: Preparing
98b5f35ea9d3: Preparing
63d7ce983cd5: Waiting
b33db0c3c3a8: Waiting
98b5f35ea9d3: Waiting
 + ansible-playbook playbook.yml

PLAY [Playbook 과 Jenkins를 이용한 CICD] ********************************************

TASK [Gathering Facts] ***************************************************************
ok: [[redacted]]

TASK [Master에게 yml file 복사] ******************************************************
ok: [[redacted]]

TASK [deployment 배포] ***************************************************************
[WARNING]: Consider using 'become', 'become_method', and 'become_user' rather
than running sudo
changed: [[redacted]]

TASK [Services 목록 출력] ************************************************************
changed: [[redacted]]

TASK [배포한 Deployment 공인 IP 확인] ************************************************
ok: [[redacted] => {
    "svc_output.stdout": "NAME         TYPE         CLUSTER-IP      EXTERNAL-IP     PORT(S)        AGE\ndoro-svc     LoadBalancer [redacted] 80:30582/TCP   3m6s\nkubernetes   ClusterIP    [redacted]   <none>          443/TCP        4d2h"
}
```

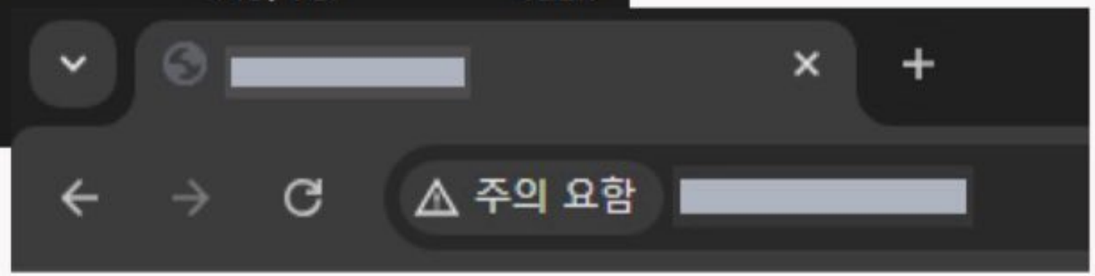
LoadBalancer [redacted] 80:30582/TCP 3m6s\nkubernetes ClusterIP [redacted]

07 구성파일 & 작동 in Jenkins

```
root@Master2:~# k get pods,svc,secret
NAME                              READY   STATUS    RESTARTS   AGE
pod/doro-dp-bc44f5559-9kph2       1/1     Running   0          5m28s
pod/doro-dp-bc44f5559-rfcm2       1/1     Running   0          5m28s
pod/doro-dp-bc44f5559-wfrl9       1/1     Running   0          5m28s

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/doro-svc     LoadBalancer   [redacted]      [redacted]    80:30582/TCP   5m28s
service/kubernetes   ClusterIP      [redacted]      <none>        443/TCP        4d2h

NAME                   TYPE                                  DATA
secret/dockerauth      kubernetes.io/dockerconfigjson        1
```



DORO PAGE

08 Grafana & Prometheus

- 실시간 데이터를 시각적으로 표현하며, 대시보드에서 시스템 상태를 쉽게 모니터링
- Prometheus는 다수의 서비스를 손쉽게 모니터링할 수 있는 확장성을 제공하며, Grafana는 다양한 데이터 소스를 통합

